

Unit_IV_chapter_II

WEB Programming: Introduction, Web Surfing with Python,
Creating Simple Web Clients, Advanced Web Clients, CGI-
Helping Servers Process Client Data, Building CGI Application
Advanced CGI, Web (HTTP) Servers.

Introduction to Web Programming

1. What is Web Programming?

- Web programming is about writing code that makes computers talk to each other over the **internet**.
- **Client**: The computer or device that requests information (e.g., your browser like Chrome).
- **Server**: The computer that stores information (websites, apps) and sends it back when asked.
- **Communication** happens using **HTTP (HyperText Transfer Protocol)**
 - think of it as the "language" of the web.

Example in real life:

- You (the **client**) walk into a restaurant and ask for food.
- The waiter takes your request to the kitchen (the **server**).
- The kitchen prepares your food and sends it back.
- That's exactly how the web works: **Client** → **Request** → **Server** → **Response** → **Client**.

2. HTTP Basics

- **HTTP** = the rules for sending/receiving web data.
- Each request contains:
 - **URL** (what page/file you want).
 - **Method** (GET = fetch info, POST = send info).
- Example:
 - <https://www.example.com/index.html>
 - **https** → protocol (secure HTTP).
 - www.example.com → server's address (domain name).
 - **index.html** → the file you want.

3. Difference between Web Client & Web Server

Role	What it Does	Example
Client	Requests info	Browser (Chrome, Firefox)
Server	Responds with info	Apache, Nginx, Flask app
Python as Client	Can fetch web pages like a browser	Using http.client or requests
Python as Server	Can host pages	Using http.server or Flask

Example Notes for Yourself:

- When I type www.example.com in my browser:
 - My browser (client) sends a request over the internet.
 - The request travels to the server that hosts www.example.com.
 - The server finds the requested page (like index.html).
 - The server send back a response (HTML code).
 - My browser reads the HTML and displays the webpage

This is the Client Server model.

```
1  import http.client
2
3  # Step 1: Connect to the server
4  conn = http.client.HTTPSConnection("www.example.com")
5
6  # Step 2: Send a request (GET / means "fetch the homepage")
7  conn.request("GET", "/")
8
9  # Step 3: Get the response
10 response = conn.getresponse()
11
12 # Step 4: Print response status and reason
13 print("Status:", response.status)
14 print("Reason:", response.reason)
15
16 # Step 5: (Optional) Print first 200 characters of the page
17 data = response.read(200).decode()
18 print("Page snippet:\n", data)
19
20 # Step 6: Close the connection
21 conn.close()
```



```
PS C:\Users\Acer> python -u "C:\Users\Acer\AppData\Local\Temp\tempCodeRunn
Status: 200
Reason: OK
Page snippet:
<!doctype html>
<html>
<head>
    <title>Example Domain</title>

    <meta charset="utf-8" />
    <meta http-equiv="Content-type" content="text/html; charset=utf-8" />
    <meta name="viewport" conten
PS C:\Users\Acer> python -u "C:\Users\Acer\AppData\Local\Temp\tempCodeRunn
Status: 200
Reason: OK
Page snippet:
<!doctype html>
<html>
<head>
    <title>Example Domain</title>

    <meta charset="utf-8" />
    <meta http-equiv="Content-type" content="text/html; charset=utf-8" />
    <meta name="viewport" conten
PS C:\Users\Acer>
```

Program Explanation (Step by Step)

- `import http.client`
 - This line imports Python's built-in module `http.client`.
 - `http.client` is like a “toolbox” that lets Python talk to web servers using HTTP/HTTPS.

Step 1: Connect to the server

```
conn = http.client.HTTPSConnection("www.example.com")
```

- Here we create a **connection object** named `conn`.
- `HTTPSConnection("www.example.com")` means:
“Hey Python, please connect securely (HTTPS) to the server at `www.example.com`.”
- Think of it like picking up the phone and dialing a website's number.

Step 2: Send a request (GET / means "fetch the homepage")

```
conn.request("GET", "/")
```

- Now that we're "connected," we send a **request**.
- "GET" is the HTTP method → it means "give me this resource."
- "/" means the **homepage** (root of the website).
- This is like saying to the waiter: *"Bring me the menu."*

Step 3: Get the response

```
response = conn.getResponse()
```

- The server replies with something (like “OK, here’s your page”).
- response now stores all that information (status, reason, content).

Step 4: Print response status and reason

```
print("Status:", response.status)
```

```
print("Reason:", response.reason)
```

response.status → the HTTP status code (number).

- 200 = OK (everything worked).
- 404 = Not Found (page doesn’t exist).
- 500 = Server Error.
- response.reason → a short text explanation (like OK, Not Found).

Example output:

```
Status: 200
```

```
Reason: OK
```

Step 5: (Optional) Print first 200 characters of the page

```
data = response.read(200).decode()  
print("Page snippet:\n", data)
```

- `response.read(200)` → reads **200 characters** from the server's reply.
- That reply is the **HTML code** of the webpage.
- `.decode()` → converts it from computer format (bytes) to human-readable text (string).
- Then we print it.

Example output:

Page snippet:

```
<!doctype html>  
<html>  
<head>  
  <title>Example Domain</title>
```

...

This is the **HTML source code** of the page. Browsers (like Chrome) read this and draw a webpage on the screen.

Step 6: Close the connection

```
conn.close()
```

- Finally, we **close the connection** (like hanging up the phone call).
- Always a good practice to free resources.

What You Learned Here

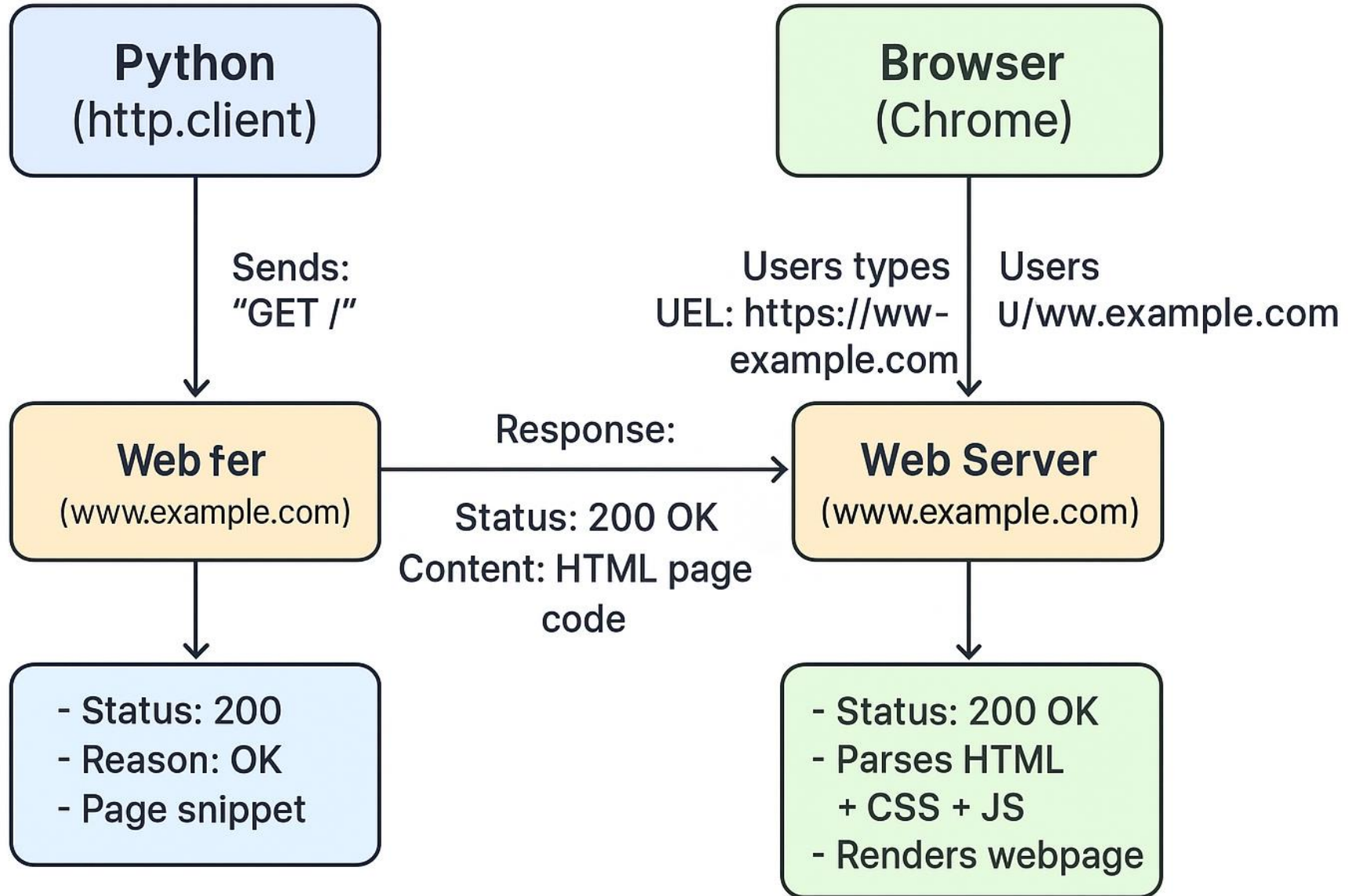
- How to **connect to a server** in Python.
- How to send a **GET request** to fetch a page.
- How to read the server's **response (status + content)**.
- That webpages are actually just **HTML text**, and browsers make it look nice.

Imagine this as:

- **You** = Python program.
- **Server** = Website's computer.
- **GET request** = Asking for the homepage.
- **Response** = Server sending HTML back.

Key Difference:

- **Python client (http.client)**
 - Only **fetches raw HTML text**.
 - Prints it out as plain text.
 - Doesn't know how to display images, styles, or JavaScript.
- **Browser (Chrome, Firefox, etc.)**
 - Fetches the same HTML text.
 - Then **interprets HTML, CSS, and JavaScript**.
 - Draws a nice-looking webpage with layout, colors, buttons, etc.



Web Surfing with Python (urllib basics)

Learn: `urllib.request.urlopen()`

- Python's **urllib** module lets you fetch data from the web (just like a browser does when you visit a site).

`urllib.request.urlopen(url)`

Opens a connection to the given URL and returns a "response object."

- You can then use `.read()` to get the data from the response.

```
1  #Lecture_2_1
2  import urllib.request
3
4  # Step 1: Open a connection to the URL
5  response = urllib.request.urlopen("http://example.com")
6
7  # Step 2: Read the entire page contents (bytes)
8  html_data = response.read()
9
10 # Step 3: Decode bytes to string (UTF-8 text)
11 html_text = html_data.decode("utf-8")
12
13 # Step 4: Print first 500 characters to see what we got
14 print(html_text[:500])
```

Program Explanation (Line by Line)

```
import urllib.request
```

This imports Python's urllib.request module.

- **urllib** = URL library (helps you talk to websites).
- **request** = part of that library that lets you send HTTP requests.

```
# Step 1: Open a connection to the URL
```

```
response = urllib.request.urlopen("http://example.com")
```

Step 1: Open a connection to the URL

```
response = urllib.request.urlopen("http://example.com")
```

- **Connects to the website** `http://example.com`.
- Think of it as dialing the server and saying:
- "Hi, I want to see the homepage."
- It returns a **response object** (like an envelope containing the server's reply).
- This object contains the webpage data, status code (like 200 OK), and headers.

Step 2: Read the entire page contents (bytes)

```
html_data = response.read()
```

- `.read()` downloads the entire webpage source code.
- The data is in **bytes** (raw computer format, not yet human-readable text).
- Example: `b'<!doctype html>....'`

Step 3: Decode bytes to string (UTF-8 text)

```
html_text = html_data.decode("utf-8")
```

Converts bytes → readable text (string).

UTF-8 is the most common way websites encode text (supports all languages and symbols).

Now you have a big string containing HTML code.

Step 4: Print first 500 characters to see what we got

```
print(html_text[:500])
```

html_text[:500] means “take the first 500 characters.”

Printing everything might be too long, so we just preview a small part.

Output will look like:

```
PS C:\Users\Acer> python -u "C:\Users\Acer\AppData\Local\Temp\tempCodeRunnerFile.python"
<!doctype html>
<html>
<head>
  <title>Example Domain</title>

  <meta charset="utf-8" />
  <meta http-equiv="Content-type" content="text/html; charset=utf-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <style type="text/css">
body {
  background-color: #f0f0f2;
  margin: 0;
  padding: 0;
  font-family: -apple-system, system-ui, BlinkMacSystemFont, "Segoe UI", "Open Sans", "Helvetica Neue", Helvetica, Arial, sans-serif;
```

```
PS C:\Users\Acer>
```

Beginner Analogy

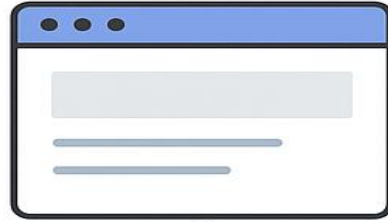
Think of this program like:

Step 1: Call a shop and ask for their catalog.

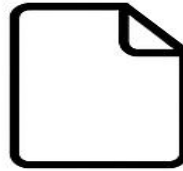
Step 2: Receive the catalog (but it's in binary — unreadable yet).

Step 3: Translate it into your language (UTF-8 decoding).

Step 4: Read the first few pages of the catalog to check what's inside.



`http://example.com`
**Open connection
to the URL**

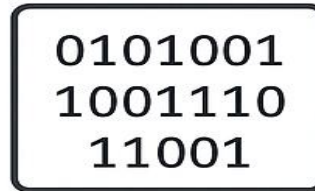


`http_response
= urlopen("http://exam-`

**http_response
contents**



`byte_data`



**Decode byte data
to text**



**First 500
characters**



{ **First 500 characters**
`<!doctype html>
<html>
<title>Example Domain</title>`

How to Verify This

- You can check it yourself in the browser:
- Open `http://example.com` in Chrome.
- Right-click → **View Page Source** (or press Ctrl+U).
- You'll see exactly the same code that Python printed.

Save to a File and Open in Browser

```
1  #Lecture_2_2
2  import urllib.request
3  import webbrowser
4
5  url = "http://example.com"
6
7  # Step 1: Fetch page
8  response = urllib.request.urlopen(url)
9  html_data = response.read().decode("utf-8")
10
11 # Step 2: Save to a local file
12 with open("example_saved.html", "w", encoding="utf-8") as file:
13     file.write(html_data)
14
15 print("Page saved as example_saved.html")
16
17 # Step 3: Open in your default browser
18 webbrowser.open("example_saved.html")
```

Understanding the Code

```
import urllib.request  
import webbrowser
```

- **urllib.request** → A Python module that lets your code talk to websites (send requests, download web pages, etc.).
- **webbrowser** → A Python module that can open URLs or local files in your computer's default browser (like Chrome or Edge).

url = "http://example.com"

- Here we store the website address in a variable called url.
- This makes it easy to reuse later, instead of typing the address again and again.

Step 1: Fetch page

```
response = urllib.request.urlopen(url)
```

```
html_data = response.read().decode("utf-8")
```

- `urllib.request.urlopen(url)` → **Opens a connection to the website** and asks the server for the page.
- The server sends back data (called a **response**).
- `response.read()` → Reads all the data from the response as **raw bytes**.
- `.decode("utf-8")` → Converts those bytes into **readable text** (HTML) using UTF-8 encoding (the most common text format on the web).
- At this point, `html_data` is just a big string containing the page's HTML code — exactly what your browser sees.

Step 2: Save to a local file

with open("example_saved.html", "w", encoding="utf-8") as file:

file.write(html_data)

- with open(..., "w", ...) as file: → Opens a new file named **example_saved.html** for writing ("w" means write mode).
- file.write(html_data) → Writes the HTML we downloaded into that file.
- with → Ensures the file is **automatically closed** when you're done (no risk of leaving it half-written).
- Now you have a copy of the webpage saved **on your computer** — even if you disconnect from the internet, you can open it later.

```
print("Page saved as example_saved.html")
```

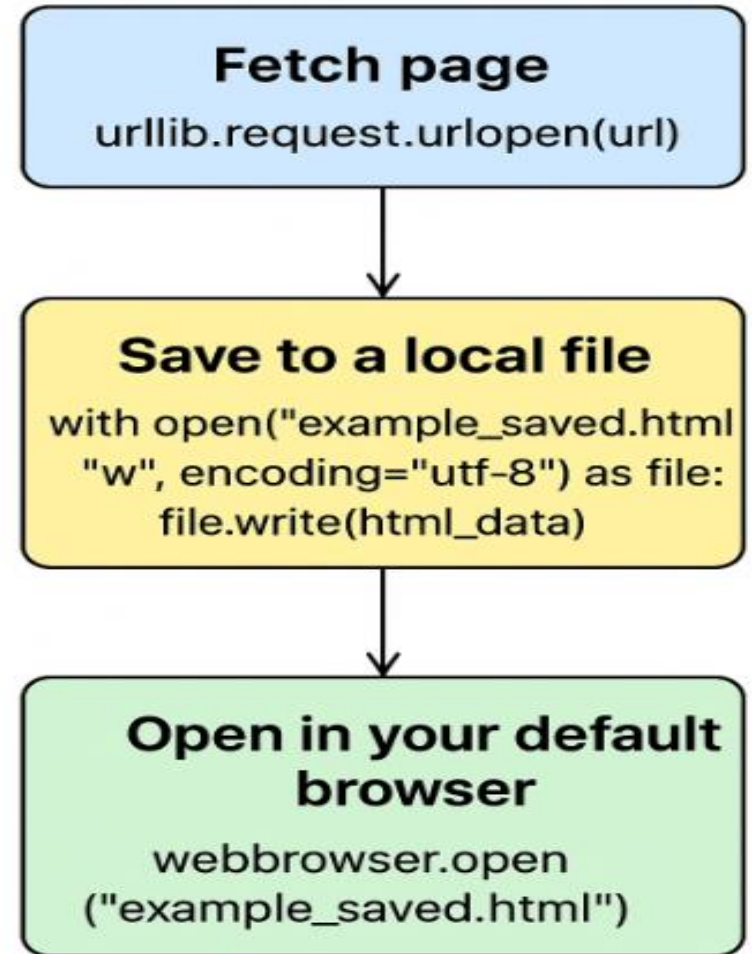
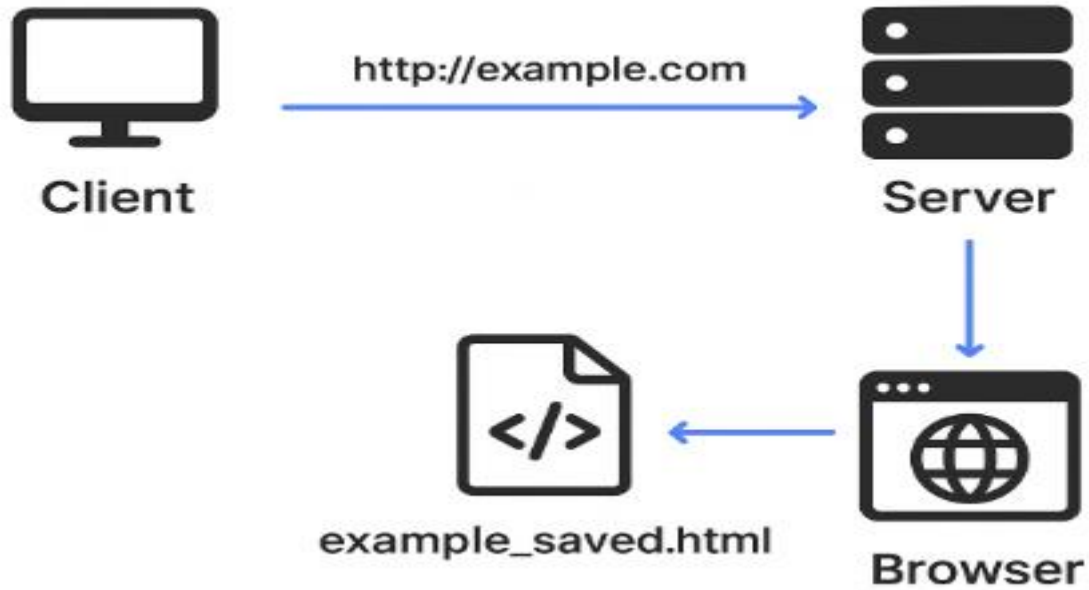
Prints a small confirmation message so you know it worked.

Step 3: Open in your default browser

```
webbrowser.open("example_saved.html")
```

This opens the file you just saved in your default browser (Chrome, Edge, Firefox, etc.).

You'll see the page exactly like it appeared online — but it's now loading from your local disk, not the internet.



Parsing Web Data (BeautifulSoup)

Why Parsing is Needed

When you fetch a webpage using `urllib.request`, you get **raw HTML code** (with `<html>`, `<head>`, `<body>` tags etc.). But most of the time, you don't need the *whole HTML* — you only need **specific data** like:

- All links (`<a href="...">`)
- Headings (`<h1>`, `<h2>`, etc.)
- Tables or lists
- Paragraph text (`<p>`)
- That's where **parsing** helps — it converts HTML into a tree-like structure so we can easily search and extract only what we want.

What is BeautifulSoup?

- **BeautifulSoup** (from the bs4 package) is a Python library that makes it easy to **parse HTML and XML documents**.

Think of it as a tool that:

- Takes raw HTML (which is just a long string),
- Converts it into a **tree-like structure**,
- Lets you search and extract exactly the data you need.

Basics of bs4.BeautifulSoup

- Here's what you need to know at the start:

1. Creating a Soup Object

- You first pass your HTML data into BeautifulSoup:

```
from bs4 import BeautifulSoup
```

```
html_doc = "<html><body><h1>Hello</h1></body></html>"
```

```
soup = BeautifulSoup(html_doc, "html.parser")
```

What this does:

This imports the BeautifulSoup class from the bs4 library.

Think of BeautifulSoup as the "tool" that will read and organize the HTML for us.

```
html_doc = "<html><body><h1>Hello</h1></body></html>"
```

What this does:

Here, html_doc is just a normal Python **string** that contains a small piece of HTML.

What is inside this string?

```
<html>
```

```
  <body>
```

```
    <h1>Hello</h1>
```

```
  </body>
```

```
</html>
```

This is a very small HTML page with:

- `<html>` → The root tag
- `<body>` → The main content section
- `<h1>Hello</h1>` → A heading that displays "Hello"

```
soup = BeautifulSoup(html_doc, "html.parser")
```

What this does:

This line creates a **BeautifulSoup object** called soup.

- Here's what happens step by step:
- **BeautifulSoup** reads the raw HTML text in html_doc.
- It converts it into a **structured tree** (kind of like a family tree of tags).
- Now you can search, extract, and manipulate parts of the HTML easily.

Visualization (What Actually Happens)

Before BeautifulSoup:

You just have a long text string:

```
"<html><body><h1>Hello</h1></body></html>"
```

After BeautifulSoup:

It becomes a tree you can explore:

```
html
```

```
└─ body
```

```
    └─ h1 → "Hello"
```

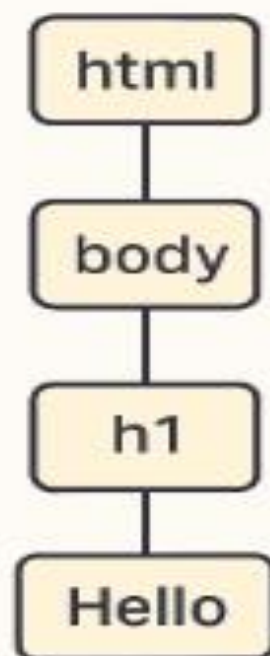
Now you can do things like:

```
print(soup.h1)      # <h1>Hello</h1>
```

```
print(soup.h1.text) # Hello
```

```
<html><body>  
  <h1>Hello</>  
</body></html>  
</html>
```

Raw HTML



Tree Structure

What "html.parser" Means

- "html.parser" tells BeautifulSoup **which tool (parser)** to use to read and understand the HTML.
- Think of it like choosing a translator — BeautifulSoup gives the HTML string to "html.parser", which **translates** it into a structured format (a tree).

Why We Need a Parser

- HTML is just text.
To extract elements like <h1>, <p>, <a>, etc., Python needs to **understand the structure** of the text.
The parser does this job.

Available Parsers

Parser Name	Description	Need to Install?	Speed
"html.parser"	Built-in Python HTML parser. Very reliable and easy for beginners.	No (comes with Python)	Medium
"lxml"	Very fast parser using lxml library.	Yes (pip install lxml)	Fast
"html5lib"	Extremely accurate, parses HTML like a real browser.	Yes (pip install html5lib)	Slow

Accessing Tags

Once you have a soup object, you can directly access tags:

```
print(soup.h1)      # <h1>Hello</h1>
```

```
print(soup.h1.text) # Hello
```

- `soup.h1` → Finds the first `<h1>` tag
- `.text` → Gives you only the text inside the tag

Finding Multiple Tags

- Use `.find_all()` to get **all matching tags**:
- `links = soup.find_all("a")` for link in links: `print(link.get("href"))` # prints all href links

Finding by Attributes

- You can filter by attributes, like class or id:
- `paragraphs = soup.find_all("p", class_="intro")`
- This finds `<p>` tags that have `class="intro"`.

Navigating the Tree

- You can move around:
- .parent → go to parent tag
- .children → iterate over child tags
- .find_next_sibling() → move to next element at same level

Common Methods

Method/Property	What It Does
.text	Get text inside a tag
.get("href")	Get an attribute value
.find(tag)	Find the first matching tag
.find_all(tag)	Find all matching tags
.select("css selector")	Search using CSS selectors

```
1  #Lecture_3_1
2  from bs4 import BeautifulSoup
3
4  html = """
5  <html><body>
6  <h1>Welcome</h1>
7  <p class="intro">Hello World!</p>
8  <a href="https://example.com">Visit Example</a>
9  </body></html>
10 """
11
12 soup = BeautifulSoup(html, "html.parser")
13
14 print("Heading:", soup.h1.text)
15 print("Paragraph:", soup.find("p", class_="intro").text)
16 print("Link:", soup.a['href'])
```

PS C:\Users\Acer> python -u "c:\Users\

Heading: Welcome

Paragraph: Hello World!

Link: https://example.com

PS C:\Users\Acer>

```
from bs4 import BeautifulSoup
```

```
Import BeautifulSoup
```

This line imports BeautifulSoup from the bs4 library, which is used to parse (read & understand) HTML easily.

```
html = """  
<html><body>  
<h1>Welcome</h1>  
<p class="intro">Hello World!</p>  
<a href="https://example.com">Visit Example</a>  
</body></html>  
"""
```

Here we are creating a string `html` that contains a small **HTML document**. This HTML has:

- A `<h1>` heading → **Welcome**
- A `<p>` paragraph → **Hello World!**
- An `<a>` link → **Visit Example**, linking to <https://example.com>
- `<a>` → Start of the link tag
- `href="https://example.com"` → **href** stands for **Hyperlink Reference** → this is the **destination URL**
- `Visit Example` → This is the **text** that users will see and click on
- `</a>` → End of the link tag

```
soup = BeautifulSoup(html, "html.parser")
```

Create a Soup Object

This line converts the raw HTML string into a **BeautifulSoup object** called soup.

Think of it like loading the HTML into a structured tree format so we can easily search and extract elements.

- **Extract <h1> Tag**

- soup.h1 → finds the first <h1> tag.

- .text → extracts just the **text inside** <h1>, ignoring HTML tags.

Output:

Heading: Welcome

```
print("Paragraph:", soup.find("p", class_="intro").text)
```

Find Paragraph by Class Name

- `soup.find("p", class_="intro")` → finds a `<p>` tag with `class="intro"`.
- `.text` → extracts its text.

Output:

- Paragraph: Hello World!


```
print("Link:", soup.a['href'])
```

Get Link (href Attribute)

soup.a → finds the first <a> tag.

['href'] → fetches the value of the href attribute (the actual URL).

Output:

Link: <https://example.com>

Code Part	What It Does	Example Output
soup.h1.text	Gets text inside <h1>	Welcome
soup.find("p", class_="intro")	Finds <p> with class intro	Hello World!
soup.a['href']	Gets link from first <a> tag	https://example.com

Extract All <a> Links from a Webpage

```
1  #Lecture_3_2
2  import urllib.request
3  from bs4 import BeautifulSoup
4
5  url = "https://www.python.org"
6
7  # Step 1: Fetch page
8  response = urllib.request.urlopen(url)
9  html_data = response.read()
10
11 # Step 2: Parse HTML
12 soup = BeautifulSoup(html_data, "html.parser")
13
14 # Step 3: Extract all links
15 links = soup.find_all("a")
16
17 print("All links on the page:\n")
18 for link in links:
19     href = link.get("href")
20     if href:
21         print(href)
```

```
import urllib.request  
from bs4 import BeautifulSoup
```

urllib.request → A Python module that lets us connect to websites and download pages.

BeautifulSoup (from bs4) → A library that helps us read and search HTML easily.

```
url = "https://www.python.org"
```

We store the website address (URL) in a variable called url.
This is the page we want to fetch and read.

Step 1: Fetch page

```
response = urllib.request.urlopen(url)
```

```
html_data = response.read()
```

Step 1 – Fetch the Webpage

`urllib.request.urlopen(url)` → connects to <https://www.python.org>.

`response.read()` → downloads the entire page's source code (HTML) into memory as bytes.

At this stage, `html_data` just contains raw HTML text.

Step 2: Parse HTML

```
soup = BeautifulSoup(html_data, "html.parser")
```

Step 2 – Parse the HTML

This line converts the raw HTML into a BeautifulSoup object called soup.

Now soup behaves like a tree you can search through (for tags like <a>, <h1>, etc.).

Step 3: Extract all links

```
links = soup.find_all("a")
```

Step 3 – Find All <a> Tags

<a> is the HTML tag used to create hyperlinks.

.find_all("a") searches the entire page and gives you a list of all <a> tags.

Example of an <a> tag:

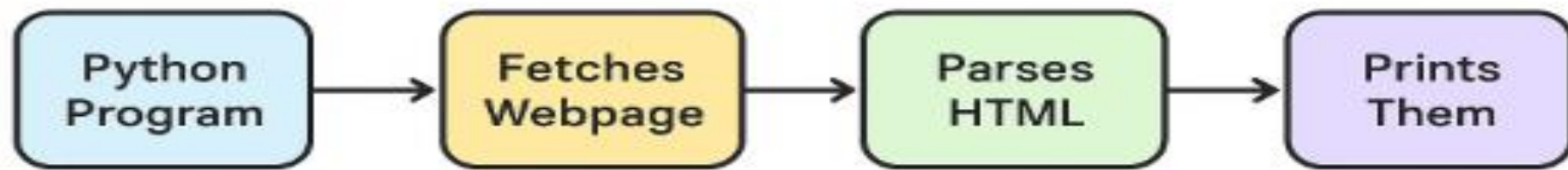
- `<a href="https://docs.python.org">Docs</a>`
- `print("All links on the page:\n")` for link in links: href = link.get("href") if href: print(href)

Step 4 – Loop and Print Links

- `for link in links:` → loops through every `<a>` tag we found.
- `link.get("href")` → extracts the value of href attribute (the actual URL or link target).
- `if href:` → makes sure it's not `None` (sometimes links might be empty).
- `print(href)` → prints each link on a new line.

Example Output (Partial)

- When you run this, you will see many links printed, like:
- /
- /about/
- /downloads/
- /community/ https:/
- /docs.python.org https:/
- /jobs.python.org
- These are all the links available on <https://www.python.org>.



Why This is Useful

- This is the first step toward **web scraping**.
You can:
- Collect links for further analysis.
- Build scripts that visit and download multiple pages. (**Automatically goes to more than one webpage** (like a human clicking links but done by code)).
- **Fetches the HTML content** of each page one by one.
- **Saves or processes that data** — for example, storing the HTML in files, extracting text, or collecting links.)
- Extract only the data you care about.

```
1  #Lecture_3_3
2  import urllib.request
3
4  # List of pages we want to visit
5  pages = [
6      "http://example.com",
7      "https://www.python.org"
8  ]
9
10 for i, url in enumerate(pages, start=1):
11     print(f"Visiting: {url}")
12
13     # Step 1: Open the page
14     response = urllib.request.urlopen(url)
15
16     # Step 2: Read and decode HTML
17     html = response.read().decode("utf-8")
18
19     # Step 3: Save to a file
20     filename = f"page_{i}.html"
21     with open(filename, "w", encoding="utf-8") as file:
22         file.write(html)
23
24     print(f"Saved {url} as {filename}\n")
25
26 print("All pages downloaded successfully!")
```

We make a list of URLs

- `pages = ["http://example.com", "https://www.python.org"]`

So our program knows which websites to visit.

We loop through each URL

- `for i, url in enumerate(pages, start=1):`
- This lets us handle multiple pages one by one.

We open the page & read its HTML

- `response = urllib.request.urlopen(url)` `html = response.read().decode("utf-8")`

This is like opening the page in a browser and copying its source code.

We save it to a file

- `with open(filename, "w", encoding="utf-8") as file: file.write(html)`
- Now you can double-click the file and open it in a browser anytime!

Python Program



Fetches Webpage



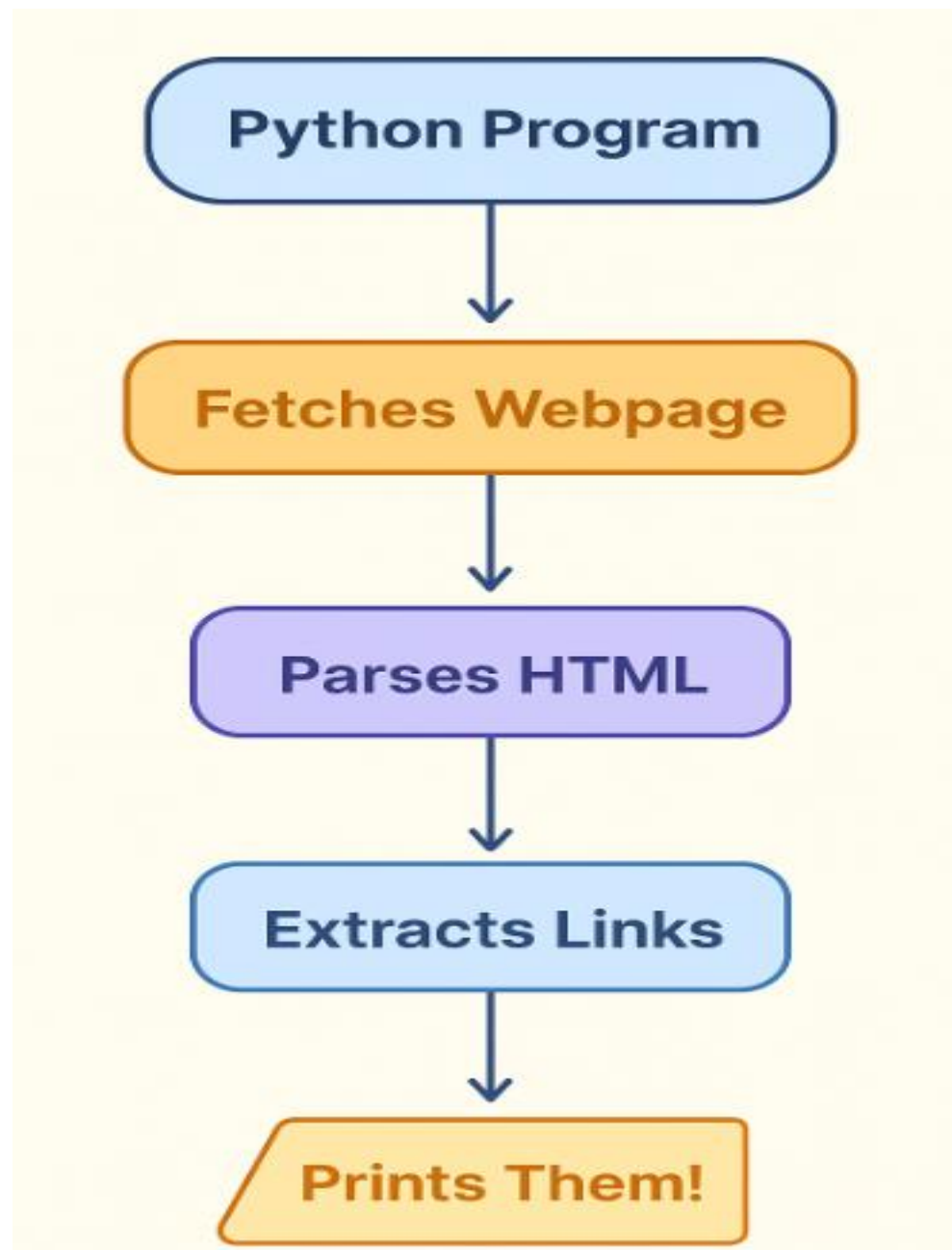
Parses HTML



Extracts Links



Prints Them!



Mini Project: Print All Headings (<h1>, <h2>, ...)

This program:

- **Visits** the Python official website (<https://www.python.org>).
- **Reads** the webpage's HTML code.
- **Finds all headings** (like <h1>, <h2>, ... <h6>).
- **Prints their text** in a clean, readable format.

```
1  #Lecture_3_4
2  import urllib.request
3  from bs4 import BeautifulSoup
4
5  url = "https://www.python.org"
6  response = urllib.request.urlopen(url)
7  html_data = response.read()
8
9  soup = BeautifulSoup(html_data, "html.parser")
10
11 # Find all headings h1, h2, h3, h4, h5, h6
12 for i in range(1, 10):
13     headings = soup.find_all(f"h{i}")
14     for h in headings:
15         print(f"H{i}: {h.text.strip()}")
16
```

```
import urllib.request
```

```
from bs4 import BeautifulSoup
```

- `urllib.request` → A built-in Python module to open URLs (fetch web pages).
- `BeautifulSoup` → A library that helps **parse** (understand and read) HTML easily.

```
url = "https://www.python.org"
```

- Stores the website address (URL) in a variable called url.

```
response = urllib.request.urlopen(url)
```

```
html_data = response.read()
```

- `urlopen(url)` → Opens a connection to the website.
- `response.read()` → Reads the entire HTML code from that page.
- The result is stored as `html_data` (in bytes form).

```
soup = BeautifulSoup(html_data, "html.parser")
```

- Creates a **BeautifulSoup object** called soup.
- "html.parser" tells BeautifulSoup to use Python's built-in HTML parser.
- Now soup acts like a tree of the webpage — we can easily find tags like <h1>, <p>, <a>, etc.

```
for i in range(1, 7):  
    headings = soup.find_all(f'h{i}')  
    for h in headings:  
        print(f'H{i}: {h.text.strip()}')
```

- for i in range(1, 7): → Loops through numbers 1 to 6 (for <h1> to <h6> tags).
- soup.find_all(f'h{i}') → Finds all heading tags of that type.
- Example: If i=1, it finds all <h1> tags.
- If i=2, it finds all <h2> tags, etc.
- for h in headings: → Goes through each heading found.
- h.text.strip() → Gets the **text inside the heading tag**, removes extra spaces.
- print(f'H{i}: {h.text.strip()}') → Prints the heading type and its text.
Example output might look like:

- `for i in range(1, 7):` → Loops through numbers 1 to 6 (for `<h1>` to `<h6>` tags).
- `soup.find_all(f'h{i}')` → Finds all heading tags of that type.
- Example: If `i=1`, it finds all `<h1>` tags.
- If `i=2`, it finds all `<h2>` tags, etc.
- `for h in headings:` → Goes through each heading found.
- `h.text.strip()` → Gets the **text inside the heading tag**, removes extra spaces.
- `print(f'H{i}: {h.text.strip()}')` → Prints the heading type and its text.
Example output might look like:

```
PS C:\Users\Acer> python -u "c:\Users\Acer\Desktop\
H1:
H1: Functions Defined
H1: Compound Data Types
H1: Intuitive Interpretation
H1: All the Flow You'd Expect
H1: Quick & Easy to Learn
H2: Get Started
H2: Download
H2: Docs
H2: Jobs
H2: Latest News
H2: Upcoming Events
H2: Success Stories
H2: Use Python for...
H2: >>> Python Software Foundation
PS C:\Users\Acer>
```

Advanced Web Clients (requests library)

Basic vs. Advanced Web Clients

Feature	Basic Web Client (urllib)	Advanced Web Client (requests, etc.)
Purpose	Fetch a single web page	Communicate with websites & APIs like a pro
Ease of Use	Low (more code, manual handling)	High (very simple, human-friendly syntax)
Can Handle	Simple GET requests	GET, POST, PUT, DELETE, headers, cookies, sessions
Error Handling	Manual	Built-in support
Data Format	Raw HTML/Text only	HTML, JSON, files, streaming, authentication

Why Do We Need Advanced Web Clients?

- Because modern websites and apps don't just send plain HTML — they send **data in JSON**, expect **authentication**, use **cookies**, and require **more than just one request**.

Advanced web clients allow us to:

- **Fetch data from APIs** (weather data, stock prices, news feeds, etc.).
- **Send data to servers** (submit forms automatically).
- **Handle sessions** (stay logged in across requests).
- **Download multiple pages/files easily**.
- **Work with modern data formats** like JSON directly.

Examples of Advanced Web Clients:

requests library in Python

- It's **simpler** and more **human-friendly**.
- Handles things like encoding, redirects, and errors automatically.
- Makes sending data (POST requests) very easy.

Installing and Using requests:

The requests library is **not built-in**, so first install it (only once):

pip install requests

Then use it in Python:

import requests

Sending a GET Request:

GET requests are like **reading** data from a website or API.

Example:

```
1  #Lecture_4_1
2  import requests
3
4  url = "https://jsonplaceholder.typicode.com/posts/1"
5  response = requests.get(url)
6
7  print("Status Code:", response.status_code)
8  print("Response Data:", response.json()) # Convert JSON into a Python dictionary
```


import requests

- We are **importing the requests library** — one of the most popular Python libraries for web communication.
- Think of it as your "web messenger" that can talk to websites or APIs.

url = <https://jsonplaceholder.typicode.com/posts/1>

Part	Meaning
https://	The protocol — tells Python to use secure HTTP (HTTPS).
jsonplaceholder.typicode.com	The domain name — this is the server we are contacting.
/posts/1	The path — tells the server what data we want. Here, we are asking for the post with ID = 1.

THINK OF IT LIKE.

- **Domain** → The building where the data lives.
- **/posts/1** → The specific room inside the building where "Post #1" is kept.

response = requests.get(url)

- requests.get() sends a **GET request** to the given URL.
- The server at jsonplaceholder.typicode.com receives our request and sends back some **data**.
- That data is stored in response.

```
print("Status Code:", response.status_code)
```

- `response.status_code` tells us if the request was successful.

- Common codes:

200 → Success

404 → Page not found

500 → Server error

Printing this helps you **debug** whether the request worked.

```
print("Response Data:", response.json())
```

1. response

- This is the object you got back from `requests.get(url)`. It contains:
- The **status code** (e.g., 200 means success)
- The **body** of the response (the data sent by the server)

2. .json()

- This is a **method** provided by the requests library.
- It **takes the response body** (which is in **JSON format** — a popular format for sending data between computers)
- And **converts it into a Python object** (usually a dictionary).

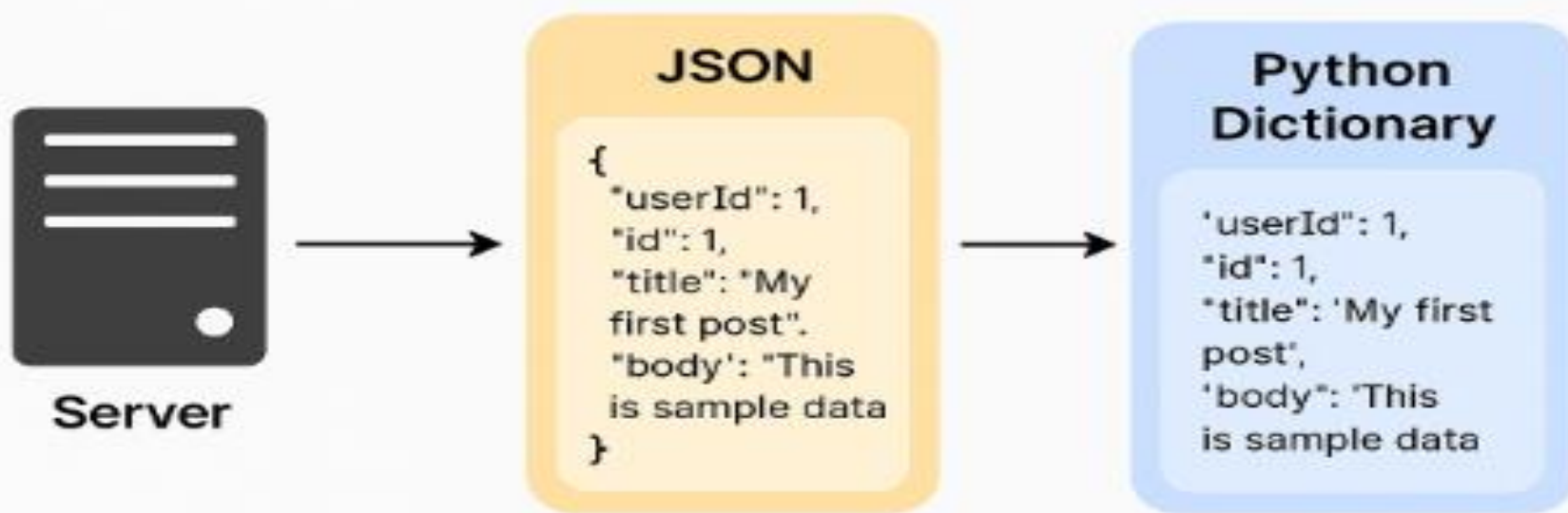
So if the server sent:

```
{  
  "userId": 1,  
  "id": 1,  
  "title": "My first post",  
  "body": "This is sample data"  
}
```

Then `response.json()` turns it into this Python dictionary:

```
{'userId': 1, 'id': 1, 'title': 'My first post', 'body': 'This is sample data'}
```

JSON to Python Dictionary



Sending a POST Request

- POST requests are like **sending data** (e.g., submitting a form).

Example:

```
#Lecture_4_2
import requests

url = "https://jsonplaceholder.typicode.com/posts"
data = {"title": "New Post", "body": "This is a test", "userId": 1}

response = requests.post(url, json=data)

print("Status Code:", response.status_code)
print("Created Post:", response.json())
```

Step-by-step Explanation

```
import requests
```

Imports the requests library – this is a very popular Python library used for sending HTTP requests (GET, POST, PUT, DELETE, etc.).

Step-by-step Explanation

`url = "https://jsonplaceholder.typicode.com/posts"`

- **This is the endpoint (URL)** where we will send the data.
Here we are targeting /posts on the jsonplaceholder server.
Think of this like saying:
- "I want to create a new post on the server."

Step-by-step Explanation

`url = "https://jsonplaceholder.typicode.com/posts"`

- **This is the endpoint (URL)** where we will send the data.
Here we are targeting /posts on the jsonplaceholder server.
Think of this like saying:
- "I want to create a new post on the server."

```
data = {"title": "New Post", "body": "This is a test", "userId": 1}
```

- **This is the data we want to send.**
It's written as a Python dictionary.
- It means:
- **title** → "New Post"
- **body** → "This is a test"
- **userId** → 1
- This is the information that will become our new post on the server.

```
response = requests.post(url, json=data)
```

- **This sends a POST request** to the server.
- `requests.post()` is used when you want to **send data to create something new** (like a new post, new user, etc.).
- `json=data` automatically converts your Python dictionary into JSON format (the format that servers understand).

```
response = requests.post(url, json=data)
```

- **This sends a POST request** to the server.
- `requests.post()` is used when you want to **send data to create something new** (like a new post, new user, etc.).
- `json=data` automatically converts your Python dictionary into JSON format (the format that servers understand).

```
print("Status Code:", response.status_code)
```

Prints the HTTP status code – this tells you whether your request succeeded or failed.

- 201 → means "Created successfully"
- 200 → means "OK"
- 400 → means "Bad request" (something went wrong)


```
print("Created Post:", response.json())
```

This converts the response (which is JSON) back into a Python dictionary and prints it. You'll see something like:

```
PS C:\Users\Acer> python -u "c:\Users\Acer\Desktop\Modulie_IV_Chapter_II\Lecture_4\Lec
Status Code: 201
Created Post: {'title': 'New Post', 'body': 'This is a test', 'userId': 1, 'id': 101}
PS C:\Users\Acer>
```

the created data (plus a new ID) → you print the result.

Fetch JSON data from <https://jsonplaceholder.typicode.com/posts/1>

```
1  #Lecture_4_3
2  import requests
3  url = "https://jsonplaceholder.typicode.com/posts/1"
4  response = requests.get(url)
5  if response.status_code == 200:
6      post = response.json()
7      print(post)
8      #print("Title:", post["title"])
9      #print("Body:", post["body"])
10 else:
11     print("Failed to fetch data")
```

This program **fetches a post (article)** from a fake online API (JSONPlaceholder) and prints its **title** and **body**.

Step-by-Step Explanation

```
import requests
```

Imports the requests library – This is a popular Python library used to send HTTP requests (GET, POST, etc.) to websites and APIs.

url = "https://jsonplaceholder.typicode.com/posts/1"

Stores the URL of the API you want to get data from.

Here, /posts/1 means “Get the first post”.

response = requests.get(url)

- **Sends a GET request** to the server.
- `requests.get(url)` asks the server to send back the data at that address.
- The server sends back a **response** (like a reply to your question).
- `response` stores this reply.

if response.status_code == 200:

- **Checks if the request was successful.**
- response.status_code tells you the status of your request.
- 200 means "OK" (success).

post = response.json()

- Converts the **JSON data** from the server into a **Python dictionary**.
Now you can access its keys like `post["title"]` or `post["body"]`.

```
print("Title:", post["title"])  
print("Body:", post["body"])
```

Prints the title and body of the post in a user-friendly way.

else:

```
    print("Failed to fetch data")
```

If the status code is **not 200**, it means something went wrong (like no internet or server issue), so it shows an error message.


```
print("Title:", post["title"])  
print("Body:", post["body"])
```

Prints the title and body of the post in a user-friendly way.

else:

```
    print("Failed to fetch data")
```

If the status code is **not 200**, it means something went wrong (like no internet or server issue), so it shows an error message.

Quick Example of Output:

```
PS C:\Users\Acer> python -u "c:\Users\Acer\Desktop\Modulie_IV_Chapter_II\Lecture_4
Title: sunt aut facere repellat provident occaecati excepturi optio reprehenderit
Body: quia et suscipit
suscipit recusandae consequuntur expedita et cum
reprehenderit molestiae ut ut quas totam
nostrum rerum est autem sunt rem eveniet architecto
PS C:\Users\Acer>
```

The **output looks like that** because of two main reasons:

The API (JSONPlaceholder) Decides What to Send

- The URL `https://jsonplaceholder.typicode.com/posts/1` points to a **fake online database** (used for learning/testing).
- When you request `/posts/1`, the server sends you a "post" (like a blog article).
- The **title** and **body** you see in the output are **predefined sample data** stored on the server.
- So the output will **always be the same**, no matter when or where you run the code.

You Printed Specific Keys

In your program, you wrote:

- `print("Title:", post["title"]) print("Body:", post["body"])`
- `post["title"]` → Shows the **title** stored in that JSON data.
- `post["body"]` → Shows the **body** text stored in that JSON data.
- If you printed `print(post)` directly, you would see the **full dictionary**:
- But you are only showing the **title** and **body**, so you get that specific output.

Why That Text Looks Weird

- The words look like Latin because it's **placeholder text** (like Lorem Ipsum).
- This API is meant for practice, not real news articles — so it uses dummy text.

Mini Project – Fetch and Print All Post Titles

```
1  #Lecture_4_4
2  import requests
3  url = "https://jsonplaceholder.typicode.com/posts"
4  response = requests.get(url)
5  if response.status_code == 200:
6      posts = response.json()
7      print("All Post Titles:\n")
8      for post in posts:
9          print(f"- {post['title']}")
10 else:
11     print("Error fetching posts")
```

Importing the Library

- import requests
- We use the **requests** library because it makes it very easy to work with the web.
- It lets us send HTTP requests (like GET, POST, etc.) and receive responses.

Setting the URL

```
url = "https://jsonplaceholder.typicode.com/posts"
```

- This is the **API endpoint** (like a special web address) that returns **a list of 100 posts**.
- Each post has an id, userId, title, and body.

Fetching Data

```
response = requests.get(url)
```

- This sends a **GET request** to the server and stores the server's response in response.

Checking for Success

if response.status_code == 200:

- Every response has a **status code**.
- 200 means **OK** (successful request).
- If it's not 200, we print an error message.

Converting JSON → Python

```
posts = response.json()
```

- The server sends data in **JSON format**.
- `response.json()` converts it into a **Python list of dictionaries**.
- Each **dictionary** represents one post.

Printing Titles

```
print("All Post Titles:\n")
```

```
for post in posts:
```

```
    print(f"- {post['title']}")
```

- posts is a list, so we loop through it using for post in posts.
- Each post is a dictionary, so post['title'] gives us the title of that post.
- We print all the titles in a neat bullet-point style.

Else Case

else:

```
print("Error fetching posts")
```

- If something goes wrong (like no internet or server error), this message is shown.

Example Output

- When you run this program, you'll get a list of 100 titles like:

```
PS C:\Users\Acer> python -u "c:\Users\Acer\Desktop\Modulie_IV_Chapter_II\Lectur
All Post Titles:
```

- sunt aut facere repellat provident occaecati excepturi optio reprehenderit
- qui est esse
- ea molestias quasi exercitationem repellat qui ipsa sit aut
- eum et est occaecati
- nesciunt quas odio
- dolore eum magni eos aperiam quia
- magnam facilis autem
- dolore dolore est ipsam
- nesciunt iure omnis dolore tempora et accusantium
- optio molestias id quia eum
- et ea vero quia laudantium autem
- in quibusdam tempore odit est dolore
- dolorum ut in voluptas mollitia et saepe quo animi
- voluptatem eligendi optio

In Nutshell

- Connected to a public API
- Downloaded **multiple posts at once**
- Extracted **only the titles**
- Printed them in a nice, human-readable list

Errors (when things go wrong) and
Headers (extra information you must send with requests).

1. Handling HTTP Errors (404, 500, etc.)

When you make a request to a server, the server always replies with a **status code**.

Examples:

Status Code	Meaning	Example
200	OK (Everything is fine)	Page loaded successfully
404	Not Found	The page doesn't exist
500	Internal Server Error	Server crashed while processing request
403	Forbidden	You don't have permission to access the resource

```
1  #Lecture_5_1
2  import requests
3
4  url = "https://jsonplaceholder.typicode.com/unknownpage"
5  response = requests.get(url)
6
7  if response.status_code == 200:
8      print("Website is OK!")
9  else:
10     print(f" Website returned status: {response.status_code}")
11
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```
PS C:\Users\Acer> python -u "c:\Users\Acer\Desktop\Modulie_IV_Chapt
Website returned status: 404
PS C:\Users\Acer>
```

2. Custom Headers & User-Agents

- Sometimes, servers block requests if they don't look like they are coming from a **real browser**.

A **User-Agent** is like an **ID card** your browser sends saying:
"Hi! I'm Chrome on Windows, please serve me a page!"

```
1 #Lecture_5_2
2 import requests
3
4 url = "https://httpbin.org/headers"
5
6 headers = {
7     "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) Chrome/114.0"
8 }
9
10 response = requests.get(url, headers=headers)
11
12 print("Server received headers:\n", response.json())
```

PS C:\Users\Acer> python -u "c:\Users\Acer\Desktop\Modulie_IV_Chapter_II\Lecture_5\#Lecture_5_1.py"

Server received headers:

```
{'headers': {'Accept': '*/.*', 'Accept-Encoding': 'gzip, deflate', 'Host': 'httpbin.org', 'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) Chrome/114.0', 'X-Amzn-Trace-Id': 'Root=1-68b9c269-6d85689120fcd71d371b2f90'}}
```

PS C:\Users\Acer>

import requests

Step 1: Import requests library

- requests is a very popular Python library that makes it easy to send HTTP requests (like GET, POST) to websites.
- It is simpler than urllib and widely used by professionals.

```
url = "https://httpbin.org/headers"
```

Step 2: Choose the URL

- `https://httpbin.org/headers` is a **test website** designed for developers.
- When you visit this URL, it shows you all the **headers** your browser/program sent to the server.
- Perfect for learning because you can see exactly what your program is doing.

```
headers = {  
    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64)  
    Chrome/114.0"  
}
```

Step 3: Create a custom header

- A **header** is extra information sent along with your request.
- Here we set "User-Agent" → which is like an **ID card** telling the server who we are.
- "Mozilla/5.0 ... Chrome/114.0" is what Chrome on Windows usually sends.
- This makes our Python program look like a real browser (sometimes servers block unknown bots).

```
response = requests.get(url, headers=headers)
```

Step 4: Send a GET request

- `requests.get(url, headers=headers)` → asks the server for the page but **includes our custom headers**.
- The server receives this request, reads our "User-Agent," and sends back a response.


```
print("Server received headers:\n", response.json())
```

- **Step 5: Print what server saw**

- `response.json()` converts the server's reply (in JSON format) into a Python dictionary.

- It will show something like:

```
{  
  "headers": {  
    "Accept": "*/*",  
    "Host": "httpbin.org",  
    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) Chrome/114.0"  
  }  
}
```

```
print("Server received headers:\n", response.json())
```

- **Step 5: Print what server saw**

- `response.json()` converts the server's reply (in JSON format) into a Python dictionary.

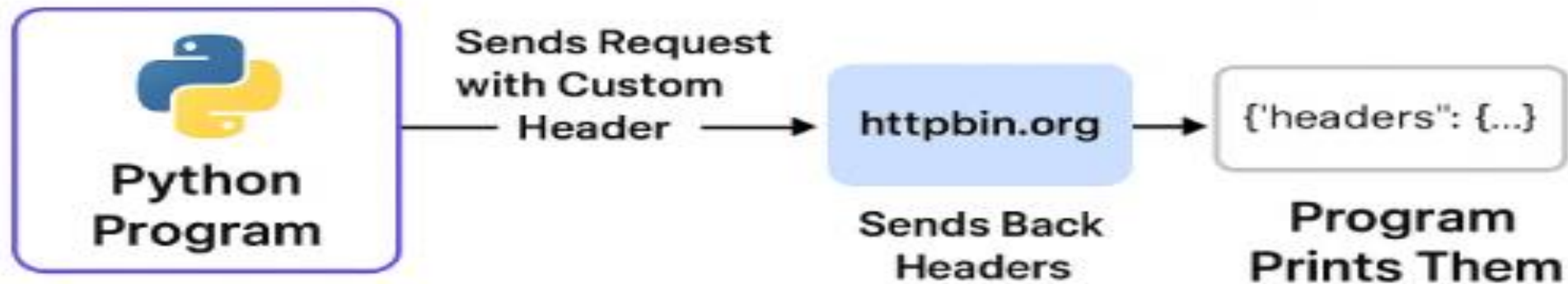
- It will show something like:

```
{  
  "headers": {  
    "Accept": "*/*",  
    "Host": "httpbin.org",  
    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) Chrome/114.0"  
  }  
}
```

You can clearly see your custom User-Agent in the output — this means the server accepted it.

- **In Simple Words**

- This program is like sending a letter to a server that says:
"Hi, I am Chrome browser (not a Python bot). Please show me what headers you received."
And the server replies:
"Sure! Here is a copy of what I got from you."



- Write a small script to send a request to any website with a fake User-Agent. Print status code + headers you sent.

```
1 #Lecture_5_3
2 import requests
3
4 # Choose a website (you can replace with any URL)
5 url = "https://httpbin.org/headers"
6 # Fake User-Agent to pretend we are a browser
7 headers = {
8     "User-Agent": "FakeBrowser/1.0 (Windows NT 10.0; Win64; x64)"
9 }
10 # Send a GET request with custom headers
11 response = requests.get(url, headers=headers)
12 # Print what you sent
13 print(" Request Sent with Headers:")
14 for key, value in headers.items():
15     print(f"{key}: {value}")
16 # Print status code of the response
17 print("\n Response Status Code:", response.status_code)
18 # Print what server received
19 print("\n Server Received Headers:")
20 print(response.json())
```

```
PS C:\Users\Acer> python -u "c:\Users\Acer\Desktop\Modulie_IV_Chapter_I  
I\Lecture_5\Lecture_5_3.py"
```

```
Request Sent with Headers:
```

```
User-Agent: FakeBrowser/1.0 (Windows NT 10.0; Win64; x64)
```

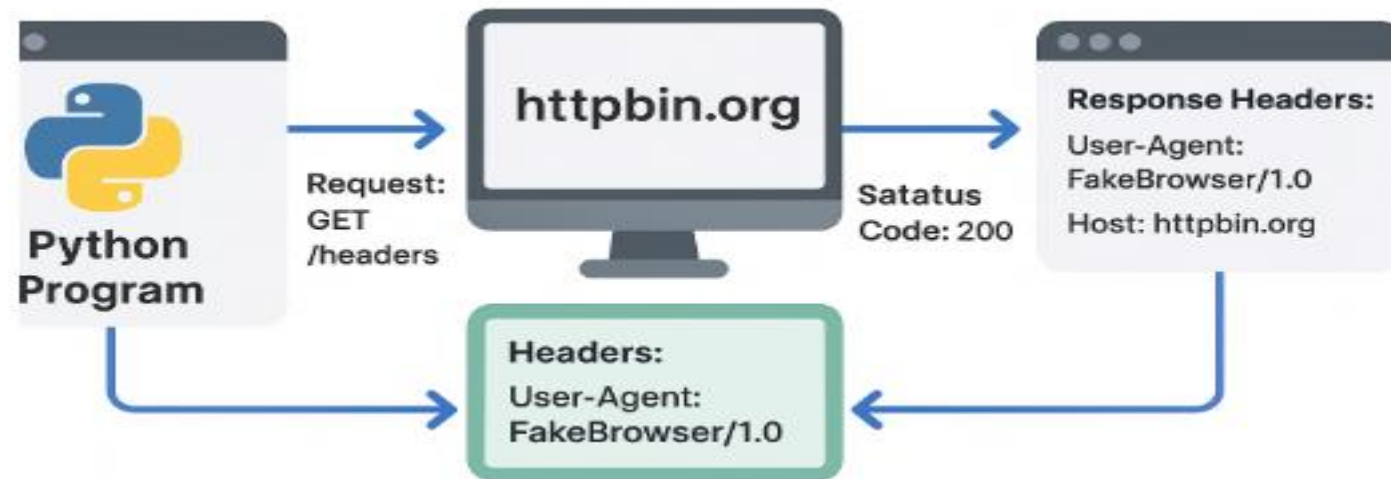
```
Response Status Code: 200
```

```
Server Received Headers:
```

```
{'headers': {'Accept': '*/*', 'Accept-Encoding': 'gzip, deflate', 'Host': 'httpbin.org', 'User-Agent': 'FakeBrowser/1.0 (Windows NT 10.0; Win64; x64)', 'X-Amzn-Trace-Id': 'Root=1-68b9c974-2e8cd7706ba06a613317c2af'}}}
```

```
PS C:\Users\Acer>
```

- Sends a request to <httpbin.org/headers> (a test site that shows what headers it received).
- Adds a **fake User-Agent** so the server thinks we are a custom browser.
- Prints the headers we sent, the status code (e.g., 200 if successful), and the headers the server saw.



Mini Project – Website Alive/Down Checker

Problem Statement:

Write a Python script that:

- Takes a list of websites.
- Checks if they are alive (200 OK) or not.
- Prints a neat summary.


```
1 #Lecture_5_4
2 import requests
3
4 websites = ["https://www.python.org", "https://www.abcxyznotrealwebsite.com"]
5
6 for site in websites:
7     try:
8         response = requests.get(site, timeout=5)
9         if response.status_code == 200:
10             print(f" {site} is alive!")
11         else:
12             print(f" {site} returned status: {response.status_code}")
13     except requests.exceptions.RequestException as e:
14         print(f" {site} is DOWN. Error: {e}")
```

```
PS C:\Users\Acer> python -u "c:\Users\Acer\Desktop\Modulie_IV_Chapter_II\Lecture_5\Lecture_5_4.py"
https://www.python.org is alive!
https://www.abcxyznotrealwebsite.com is DOWN. Error: HTTPSCConnectionPool(host='www.abcxyznotrealwebsite.com', port=443): Max retries exceeded with url: / (Caused by NameResolutionError("<urllib3.connection.HTTPSConnection object at 0x0000020081A36FC0>: Failed to resolve 'www.abcxyznotrealwebsite.com' ([Errno 11001] getaddrinfo failed)"))
```

Code Explanation

```
import requests
```

We import the requests library — this is a popular Python library used to send HTTP requests (GET, POST, etc.) to websites and APIs.

```
websites = ["https://www.python.org",  
"https://www.abcxyznotrealwebsite.com"]
```

We make a list of websites to check.

- First one is **real** (<https://www.python.org>)
- Second one is **fake** (<https://www.abcxyznotrealwebsite.com>) — this is just for testing what happens if a website does not exist.

for site in websites:

Loop through each website in the list.

This means we will run the code inside the loop once for Python.org, then again for the fake website.

try:

```
response = requests.get(site, timeout=5)
```

Try to connect to the website

- `requests.get(site)` → sends an HTTP GET request to the website.
- `timeout=5` → wait for max 5 seconds before giving up. This prevents your program from "hanging" forever if a site is too slow.

```
if response.status_code == 200:
```

```
    print(f" {site} is alive!")
```

```
else:
```

```
    print(f" {site} returned status: {response.status_code}")
```

Check the server response

- If we get **status code 200** → it means the website is working normally, so we print and say it's alive.
- If we get **any other status code** (like 404, 500), we print and display that status code.

```
except requests.exceptions.RequestException as e:
```

```
    print(f" {site} is DOWN. Error: {e}")
```

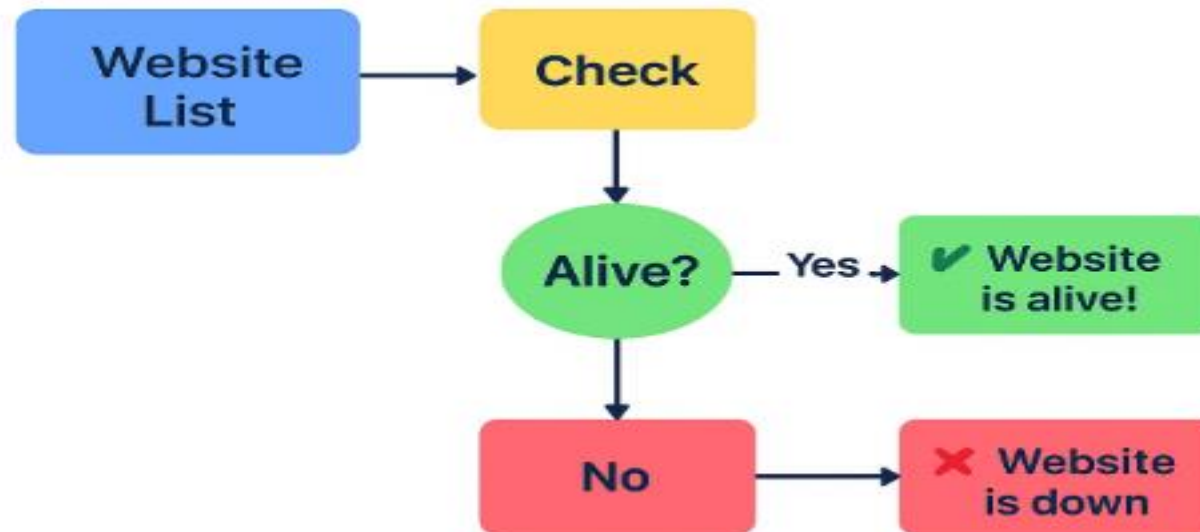
Handle errors safely

If there's a problem (site doesn't exist, no internet, timeout, etc.), Python will raise an error.

- `except ... as e:` catches that error.
- Then we print and show what the error was.

How It Works

- This program acts like a **website health checker**:
- Takes a list of websites.
- Tries to connect to each one.
- Prints if it's working, if it returns an error code, if it's completely down or unreachable.



Combining Web Client Skills Downloading Files with requests

Downloading Files with requests

- You already know how to use `requests.get()` to fetch text/JSON.
But a file (like an image, PDF, or MP3) is **binary data** – we cannot just `decode()` it like HTML.

Instead:

- We open a file in **binary write mode (wb)**.
- We write `response.content` (raw bytes) into that file.

Download an Image

- Here's a simple example:

```
1  #Lecture_6_1
2  import requests
3
4  # Step 1: Choose an image URL
5  url = "https://www.python.org/static/community\_logos/python-logo.png"
6
7  # Step 2: Send GET request
8  response = requests.get(url)
9
10 # Step 3: Check if download was successful
11 if response.status_code == 200:
12     # Step 4: Save to file (binary mode)
13     with open("python_logo.png", "wb") as file:
14         file.write(response.content)
15     print("Image downloaded successfully!")
16 else:
17     print("Failed to download image.")
```

Step 1: Choose an image URL

- `url = "https://www.python.org/static/community_logos/python-logo.png"`
- url is the **web address** of the file you want to download.
- Here, it points to the official **Python logo image** hosted on python.org.

Step 2: Send a GET request

- `response = requests.get(url)`
- `requests.get(url)` tells Python:
➡ "Hey, go to this website and **download whatever is there.**"
- The result is stored in `response`, which contains:
 - `response.status_code` → Was the download successful? (200 = OK)
 - `response.content` → The **actual file data** (in bytes).

Step 3: Check if download was successful

- if `response.status_code == 200`:
- Websites send **status codes** back.
- 200 means **OK** (everything worked fine).
- If it's not 200, maybe the file is missing (404), or the server is down (500).

Step 4: Save the file

- with open("python_logo.png", "wb") as file: file.write(response.content)
- open("python_logo.png", "wb")
 - "python_logo.png" → file name to save as (it will be in the same folder as your script).
 - "wb" → means **write binary mode** (very important for images, videos, PDFs).
- file.write(response.content) writes the raw downloaded data into the file.
- After this, you'll have a **real image file** saved on your computer.

Print a Confirmation Message

- `print("Image downloaded successfully!")`
- This is just for you to know that everything worked.
- If something went wrong, you would see:
- `print("Failed to download image.")`



Download All Images from a Webpage

- This combines our earlier **BeautifulSoup skills** + **file download skills**.

Step-by-Step Plan

- **Download a webpage**
 - Use `requests.get()`.
- **Extract all `<img>` tags**
 - Use BeautifulSoup and `soup.find_all("img")`.
- **Get the `src` attribute** of each image
 - Build full URLs if needed (sometimes they are relative paths).
- **Download each image**
 - Loop through each `src` and use `requests.get()` again.
- **Save each image to a folder**
 - Use Python's `open(..., "wb")` for each file.


```
1  #Program_6_2
2  import requests
3  from bs4 import BeautifulSoup
4  import os
5  url = "https://www.python.org"
6  response = requests.get(url)
7
8  if response.status_code == 200:
9      soup = BeautifulSoup(response.text, "html.parser")
10     images = soup.find_all("img")
11
12     # Create folder if it doesn't exist
13     if not os.path.exists("downloaded_images"):
14         os.makedirs("downloaded_images")
15     for i, img in enumerate(images, start=1):
16         src = img.get("src")
17         if src:
18             # Handle relative URLs (if they don't start with http)
19             if not src.startswith("http"):
20                 src = "https://www.python.org" + src
21             try:
22                 img_data = requests.get(src).content
23                 filename = f"downloaded_images/image_{i}.png"
24                 with open(filename, "wb") as f:
25                     f.write(img_data)
26                 print(f"Downloaded {filename}")
27             except Exception as e:
28                 print(f"Could not download {src}: {e}")
29     else:
30         print("Failed to fetch webpage.")
```

Step-by-Step Explanation

- **1. Import Required Libraries**

- `import requests from bs4 import BeautifulSoup import os`
- **requests** → allows your program to fetch web pages from the internet.
- **BeautifulSoup** → a library that helps extract specific information (like images, links) from HTML pages.
- **os** → lets you create folders or check if a folder exists on your computer.

2. Send a Request to the Website

```
url = "https://www.python.org"
```

```
response = requests.get(url)
```

- `requests.get(url)` → asks the website to send you the HTML code of the page.
- `response` → stores what the server sends back (status + page data).

3. Check if the Page Loaded Successfully

if response.status_code == 200:

- 200 means **OK** → The website responded successfully.
- If this is true, the program continues to work with the page.

4. Parse the Webpage

- `soup = BeautifulSoup(response.text, "html.parser")`
- `images = soup.find_all("img")`
- `BeautifulSoup(response.text, "html.parser")` → converts the raw HTML into an easy-to-read format.
- `.find_all("img")` → finds all `<img>` tags (images) in the page.
- `images` → is now a list of all images found on the page.

5. Create a Folder to Save Images

```
if not os.path.exists("downloaded_images"):
    os.makedirs("downloaded_images")
```

- `os.path.exists()` → checks if a folder called `downloaded_images` already exists.
- `os.makedirs()` → creates the folder if it does **not** exist.
This prevents errors when saving images later.

6. Download and Save Each Image

```
for i, img in enumerate(images, start=1):
```

```
    src = img.get("src")
```

```
    if src:
```

```
        if not src.startswith("http"):
```

```
            src = "https://www.python.org" + src
```

- **Loop through all images:** `for i, img in enumerate(images, start=1)` gives each image a number (1, 2, 3...).
- `img.get("src")` → gets the link to the image.
- If the link doesn't start with `http`, we add the website address to make it a full URL.

Download Image Data

```
img_data = requests.get(src).content
```

```
filename = f"downloaded_images/image_{i}.png"
```

- `requests.get(src).content` → fetches the actual image data (binary format).
- `filename` → creates a file name like `image_1.png`, `image_2.png`, etc.

8. Save Image to Disk

with open(filename, "wb") as f:

```
    f.write(img_data)
```

- Opens the file in **write-binary (wb) mode** (because images are not plain text).
- Writes the downloaded image data into that file.
- After this, the image is saved on your computer.

9. Error Handling

except Exception as e:

```
print(f" Could not download {src}: {e}")
```

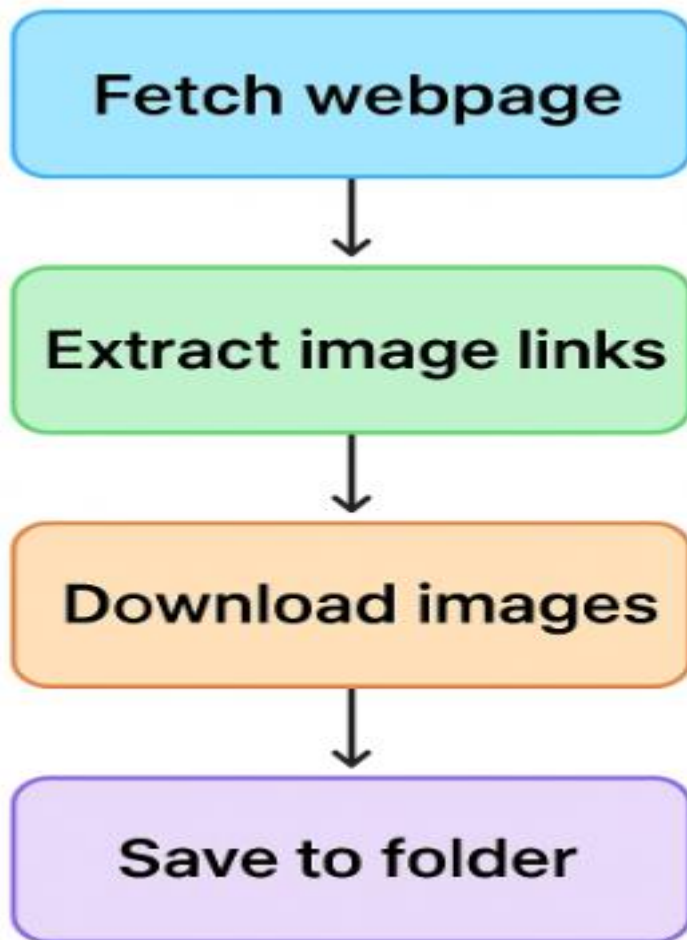
- If something goes wrong (bad URL, timeout, etc.), the program doesn't crash but shows an error message.

Final Output

It will print messages like:

- Downloaded downloaded_images/image_1.png
- Downloaded downloaded_images/image_2.png ...

You'll find all saved images inside the downloaded_images folder.



Mini Web Scraper Project

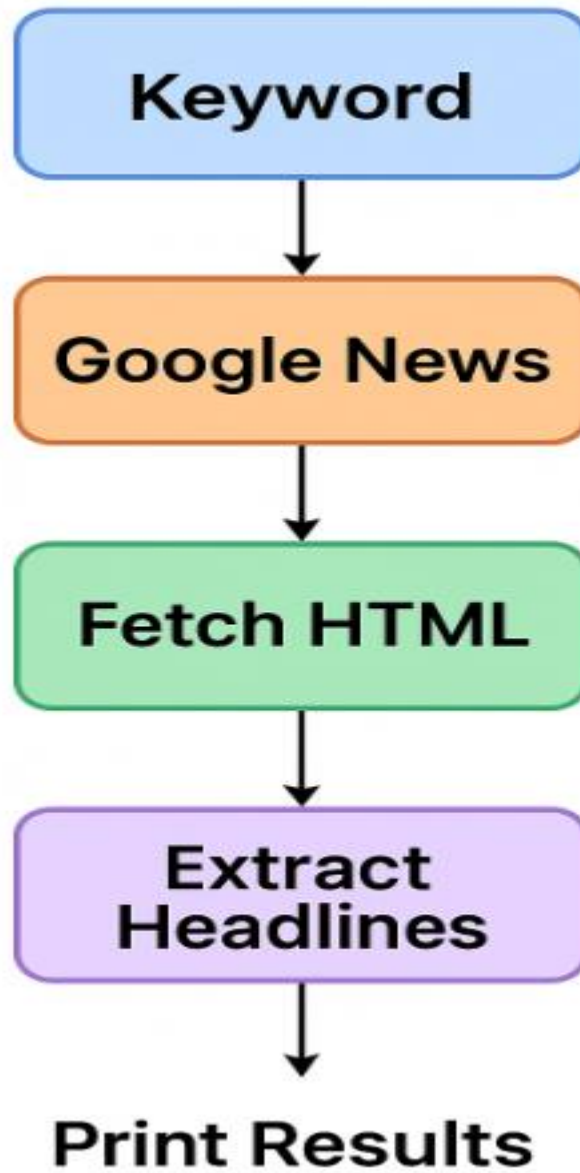
Problem Breakdown

- **Take a keyword** (e.g., "Python").
- **Search it on a news site** (like Google News).
- **Fetch top 5 headlines** from the search results.
- **Print them neatly.**

Key Skills You'll Use

- requests → To fetch the webpage.
- BeautifulSoup → To parse and extract news headlines.
- Loops & Lists → To store and print only the top 5 headlines.

```
1 #Lecture_7_1
2 import requests
3 from bs4 import BeautifulSoup
4
5 # Step 1: Take a keyword from user
6 keyword = input("Enter a keyword to search news for: ")
7
8 # Step 2: Build Google News search URL
9 url = f"https://news.google.com/search?q={keyword}"
10
11 # Step 3: Fetch page
12 response = requests.get(url)
13
14 if response.status_code == 200:
15     # Step 4: Parse HTML
16     soup = BeautifulSoup(response.text, "html.parser")
17
18     # Step 5: Extract all headline elements
19     headlines = soup.find_all("a", class_="JtKRv", limit=5)
20
21     print(f"\nTop 5 News Headlines for '{keyword}':\n")
22     for i, h in enumerate(headlines, start=1):
23         print(f"{i}. {h.text.strip()}")
24 else:
25     print("Failed to fetch news.")
```

Step-by-Step Explanation

- **Import Modules**

- requests → lets us fetch webpages.
- BeautifulSoup → helps us read & extract data from HTML.

- **Take User Input**

`keyword = input("Enter a keyword to search news: ")`

If you type "Python", the program will search for Python-related news.

Build Search URL

```
url = f"https://news.google.com/search?q={keyword}"
```

- If keyword = Python, final URL becomes:
<https://news.google.com/search?q=Python>

Fetch the Webpage

```
response = requests.get(url)
```

- Sends a request to Google News.
- Gets a response (status code 200 = success).

Parse the HTML

- `soup = BeautifulSoup(response.text, "html.parser")`

Extract Headlines

```
headlines = soup.find_all("a", class_="DY5T1d", limit=5)
```

- Finds all <a> links with class "DY5T1d" (where Google stores headline text).
- limit=5 → only top 5 headlines.

Print Headlines

- `for i, headline in enumerate(headlines, start=1): print(f'{i}. {headline.text}')`
 - Loops through each headline.
 - Prints it in a nice numbered list.

```
PS C:\Users\Acer> python -u "c:\Users\Acer\Desktop\Modulie_IV_Chapter_I
I\Lecture_7\Lecture_7_1.py"
```

```
Enter a keyword to search news for: python
```

```
Top 5 News Headlines for 'python':
```

1. How to Use Python's dataclass to Write Less Code
 2. Hands On Time Series Modeling of Rare Events, with Python
 3. Chat with data the easy way in R or Python
 4. Meet Elysia: A New Open-Source Python Framework Redefining Agentic RAG Systems with Decision Trees and Smarter Data Handling
 5. Video: 10-foot python sparks panic in Hyderabad, caught
- ```
PS C:\Users\Acer>
```

# **CGI + Building Web Servers**

**Understanding CGI, Apache, and Dynamic Content**

# What is a Web Server?

- A **web server** is both **software** and sometimes **hardware** that delivers web pages (or other web content) to users over the internet when they request it through their web browsers.



# 1. Definition

A **web server** is a system that:

- **Receives requests** from clients (usually web browsers, apps, or other programs).
- **Processes those requests** (either serving a file or executing code).
- **Sends back a response** (usually an HTML page, image, video, or data).

Think of it as a **restaurant waiter**:

- You (the browser) place an order (HTTP request).
- The kitchen (the web server software) prepares the dish (web page/data).
- The waiter delivers it back to your table (HTTP response).

## 2. Key Components

### A. Hardware

- A computer that stores website data (HTML, CSS, JS, images, videos).
- It has an IP address and domain name pointing to it.

### B. Software

- A program that understands **HTTP/HTTPS protocols**.
- Popular examples: **Apache, Nginx, Microsoft IIS, LiteSpeed**.

### 3. How a Web Server Works

- **Client Request:**

A browser sends an HTTP request (e.g., `https://example.com/index.html`).

- **Server Processing:**

The web server finds the file (or runs a script, like PHP/Python).

- **Response:**

The server sends back the requested file/data with an HTTP response.

- **Rendering:**

The browser displays the response to the user.

## 4. Types of Content Served

- **Static Content:**

Pre-made files like HTML pages, CSS, images (no processing needed).

- **Dynamic Content:**

Content generated on-the-fly (e.g., CGI scripts, PHP pages, Python Flask/Django apps).

# Static Page

- A **static page** does **not change** when you reload it.
- It is just a plain file stored on the server.
- Examples:
  - about.html
  - contact.html
- **Always shows the same content** to every user.
- **Example:**  
If you write an index.html file like this:

```
<h1>Welcome!</h1>
```

```
<p>This is my website.</p>
```

Everyone who visits will see exactly this.

# Dynamic Page

- A **dynamic page** is **generated on the fly** by a program.
- Its content can **change based on user input, database data, or time**.
- Example:
  - A Python CGI script that says:
    - `print("Hello, " + username)`
    - If you type your name as "Alex", you will see:
      - Hello, Alex
    - If another user types "Sam", they will see:
      - Hello, Sam
- So **dynamic pages are personalized**, while static pages are the same for everyone.

Feature	Static Page	Dynamic Page
Content	Fixed (same always)	Generated (changes)
Created By	HTML files	Programs (Python, PHP, etc.)
Examples	About Us page, FAQs	Search results, shopping cart
Speed	Very fast	Slightly slower (runs code)

# 5. Popular Web Servers

Web Server	Key Features	Best For
Apache HTTP Server	Open-source, widely used, supports dynamic modules	General purpose, websites
Nginx	High-performance, event-driven, reverse proxy support	High-traffic sites, load balancing
Microsoft IIS	Windows-based, GUI configuration	.NET/Windows apps
LiteSpeed	Fast, efficient, built-in caching	WordPress hosting, performance-focused



# Apache

- **Apache** is one of the most popular **web server software** in the world.
- A **web server** is a program that:
  - Waits for requests from browsers (like Chrome, Edge, Firefox).
  - Finds the requested file (or runs a program).
  - Sends the result back to the browser.

Apache can serve:

- **Static Pages** (just files like .html, .jpg, .css)
- **Dynamic Pages** (generated by programs like Python, PHP, etc.)
- Think of Apache as a **waiter in a restaurant** — it takes your order (browser request), brings the food (web page) from the kitchen (server).

## 6. Common Uses

- Hosting websites (personal blogs, e-commerce sites)
- Serving APIs for apps
- Hosting files for download
- Acting as a reverse proxy or load balancer

# What is CGI?

- CGI stands for **Common Gateway Interface**.  
It is a **bridge** between a **web server** (like Apache) and a **program** (like a Python script).
- Normally, web servers just send back **static files** (HTML, images, etc.).
- But if you want **live, changing content** (e.g., greetings, search results, forms), you need to run **code** on the server.
- CGI is the **standard way** for the web server to:
  - Receive input (like form data).
  - Run your program.
  - Send back the program's output to the browser.

# Think of CGI Like a Restaurant

- **Browser = Customer** → asks for a special dish ("What's the weather today?").
- **Web Server (Apache) = Waiter** → goes to the kitchen and passes the order.
- **CGI Script = Chef** → prepares a fresh dish (runs Python, gets real-time weather).
- **Waiter (Apache)** brings it back to the customer (browser).
- This way, every user can get **customized content** instead of a fixed menu.

# How CGI Works Step by Step

- **Browser Requests a Page**

Example: `http://localhost/cgi-bin/hello.py?name=Alex`

- **Web Server Sees It's a CGI Script**

Apache notices that `hello.py` is in `cgi-bin` folder, so it should **run it** instead of just showing its code.

- **Server Runs Your Script**

It gives input to your Python script (e.g., the name Alex from the URL).

- **Your Script Prints Output in HTML**

Example output:

Content-type: text/html

`<h1>Hello Alex!</h1>`

## **Server Sends It Back to Browser**

The browser shows **Hello Alex!** on the screen.

## Task: Simple CGI Script (hello.py)

- Here's the simplest CGI script in Python that prints **Hello World**:

```
#!/C:/Users/Acer/AppData/Local/Microsoft/WindowsApps/python.exe
^ Replace with your actual Python path (see "where python" in CMD)

print("Content-Type: text/html\n")
print("<html>")
print("<head><title>CGI Test</title></head>")
print("<body>")
print("<h1>Hello, CGI World!</h1>")
print("<p>This is Python running as a CGI script via Apache!</p>")
print("</body></html>")
```

<http://localhost:8080/cgi-bin/hello.py>

## Task: Simple CGI Script (greet.py)

- Here's the simplest CGI script in Python that prints

Enter your name:

**Hello, Abhishek!**

<http://localhost:8080/cgi-bin/greet.py>

```
1 #!C:/Users/Acer/AppData/Local/Microsoft/WindowsApps/python.exe
2 import cgi
3
4 print("Content-Type: text/html\n")
5 print("<html><body>")
6
7 form = cgi.FieldStorage()
8 name = form.getvalue("name")
9
10 if name:
11 print(f"<h1>Hello, {name}!</h1>")
12 else:
13 print("""
14 <form method="get" action="greet.py">
15 <label>Enter your name: </label>
16 <input type="text" name="name">
17 <input type="submit" value="Greet Me">
18 </form>
19 """)
20
21 print("</body></html>")
```



## **CGI Advantages**

- Easy to implement
- Language-independent (Python, Perl, C all work)
- Can generate fully dynamic HTML

## **CGI Limitations**

- Slower than modern frameworks (creates a new process per request)
- Harder to maintain for large applications
- Modern alternatives: Flask, Django, FastAPI (more efficient)

# Summary

- Apache serves static & dynamic pages.
- CGI acts as a **bridge** between browser and program.
- We can write Python scripts that output HTML.
- You learned how to make **hello.py** and a **greeting form project**.

# Handling Forms with CGI

## Learning Objectives

Understand how the cgi module works in Python.

Learn to **get data from HTML forms** and process it using a Python CGI script.

Practice creating simple **interactive web applications**.

# What is CGI?

- CGI (Common Gateway Interface) is a way for a **web server** to run a **program** and send its output back to the browser.

Think of it like this:

- **HTML form** → user types something → sends data to the server
- **CGI script** → receives data → does some work → sends result back
- In our case, we will use **Python CGI scripts**.

- Python has a built-in module called **cgi** that helps read data from HTML forms.

The most important class is:

`cgi.FieldStorage()`

- This class collects all the form data (text, numbers, radio buttons, etc.) that the user submitted.

```
<!DOCTYPE html>
<html>
<head>
 <title>CGI Form Example</title>
</head>
<body>
 <h2>Add Two Numbers</h2>
 <form action="/cgi-bin/add.py" method="get">
 Number 1: <input type="text" name="num1">

 Number 2: <input type="text" name="num2">

 <input type="submit" value="Calculate Sum">
 </form>
</body>
</html>
```

- `<form action="...">` → tells the browser where to send data (to a Python script on server).
- `method="get"` → sends data as URL parameters (you'll see it in address bar).
- `<input type="text" name="num1">` → a text box with the name **num1** (important because we use this name in Python).
- `<input type="submit">` → button to send data.

- **Create the Python CGI Script**
- Save this file inside your server's **cgi-bin** folder as **add\_numbers.py**  
(Make sure the file has execute permissions if you're on Linux.)



```
1 #!C:/Users/Acer/AppData/Local/Microsoft/WindowsApps/python.exe
2
3 import cgi
4
5 # Required HTTP header
6 print("Content-Type: text/html\n")
7
8 # Create instance of FieldStorage
9 form = cgi.FieldStorage()
10
11 # Get data from form
12 num1 = form.getvalue("num1")
13 num2 = form.getvalue("num2")
14
15 # Convert to numbers and calculate sum
16 try:
17 sum_result = int(num1) + int(num2)
18 print(f"<h2>Result: {num1} + {num2} = {sum_result}</h2>")
19 except:
20 print("<h2>Error: Please enter valid numbers.</h2>")
```

- **`#!/usr/bin/env python3`** → tells server to run script with Python 3.
- **`print("Content-Type: text/html\n")`** → must be printed first to tell browser "I'm sending HTML data".
- **`form = cgi.FieldStorage()`** → collects submitted data.
- **`form.getvalue("num1")`** → gets whatever the user typed in `name="num1"`.
- We add numbers using `int()` and print result as HTML.
- `try-except` prevents crash if user enters text instead of numbers.

Browser (form.html)

|

| [User fills numbers & clicks Submit]



Web Server (receives form data)

|

| [Executes add\_numbers.py]



Python Script (processes data)

|

| [Creates HTML response with result]



Browser (shows result page)

# Running form.html and add.py on XAMPP (Apache)

## 1. Place Your Files in Correct Folders

- **Step 1 – Open XAMPP Installation Folder**

Usually it is in:

C:\xampp\

- **Step 2 – Place form.html in htdocs folder**

This is the web root folder.

C:\xampp\htdocs\form.html

- **Step 3 – Place add\_numbers.py in cgi-bin folder**

This is the special folder where Apache expects CGI scripts.

C:\xampp\cgi-bin\add.py

## Open the Form in Browser

- Start **Apache** from XAMPP Control Panel.

- Open browser and go to:

`http://localhost:8080/form.html`

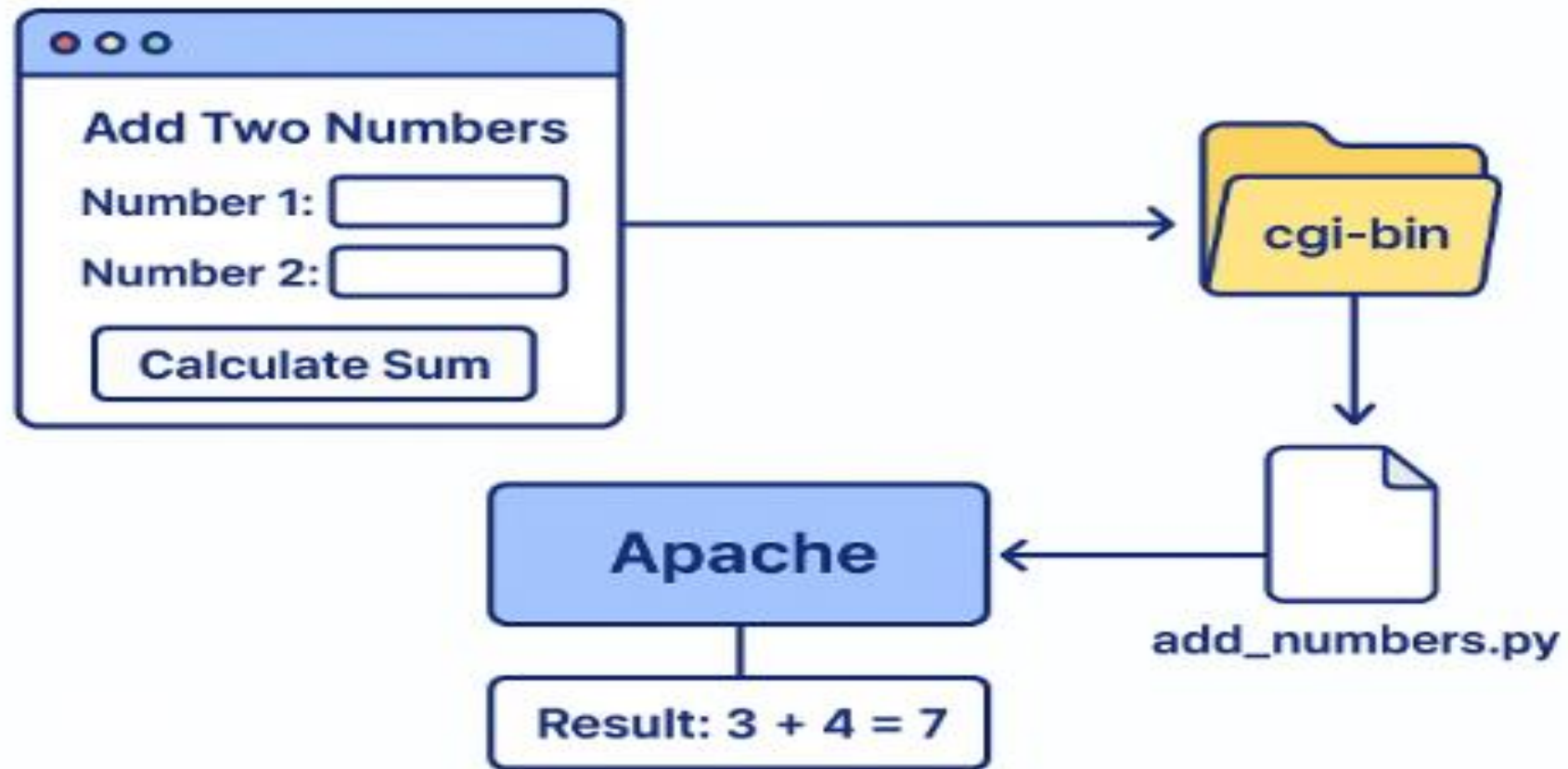
- Fill two numbers and click **Submit**.
- Apache will run `add_numbers.py` from `cgi-bin` folder and show result.

## Important Notes for add.py

- Make sure the first line is:
- `#!C:/Users/Acer/AppData/Local/Microsoft/WindowsApps/python.exe`
- This tells Apache which interpreter to use.
- Ensure file is saved with **Unix line endings (LF)** if possible (Windows works fine in most cases, but LF is safer for CGI).
- Make sure add.py has **execute permission** (not usually a problem on Windows).

## Summary:

- form.html → goes in htdocs
- add.py → goes in cgi-bin
- Enable ExecCGI and add .py handler in httpd.conf
- Access via `http://localhost:8080/form.html`





# **Advanced CGI (POST Method, Error Handling)**

**Difference between GET and POST.**

**How to handle and validate form data safely.**

**How to prevent crashes with proper error handling.**

**Built a working mini project with real-world usefulness.**

## Basic CGI (Recap)

- Simple scripts that output HTML.
- Often just print static or slightly dynamic text.
- Example: Displaying "Hello World" or current date & time.

## Advanced CGI (Meaning)

Advanced CGI focuses on **more powerful, interactive, and efficient applications** using CGI.

It typically includes:

- **Form Handling**

- Reading user input from HTML forms using `cgi.FieldStorage()`.
- Example: Login forms, search boxes, feedback pages.

- **Session Management**

- Storing information about users across multiple pages (using cookies, hidden fields, or server-side storage).
- Example: Shopping carts, logged-in user tracking.

## **Database Connectivity**

- Connecting to databases (MySQL, SQLite, PostgreSQL, etc.) to store/retrieve data.
- Example: Displaying products from a database, saving form submissions.

## **File Handling & Uploads**

- Allowing users to upload files (images, documents).
- Handling files securely on the server.

## **Dynamic Page Generation**

- Generating content based on conditions (user preferences, database queries).
- Example: Showing personalized recommendations.

## **Error Handling & Security**

- Handling invalid input safely (avoiding crashes).
- Preventing security issues like code injection, XSS, CSRF.

## **Performance Optimization**

- Using compiled CGI (FastCGI) for faster execution.
- Reducing overhead by caching frequent results.

## First, Understand What Happens in CGI

- Think of a **CGI script** like a waiter in a restaurant:
- You (the browser) fill a form → give it to the waiter.
- The waiter (CGI script) goes to the kitchen (server), processes it.
- The waiter brings back your order (HTML result page) to you.

# GET vs POST – Two Ways to Send Form Data

- When you submit a form, there are **two main ways to send the data**:

Method	How it Works	Example	When to Use
GET	Data is added to the URL	<code>http://localhost/cgi-bin/script.py?name=John</code>	For simple, small data
POST	Data is hidden inside the request body	(Nothing visible in URL)	For passwords, large text, or private data

- **POST = like whispering to the waiter** (not visible in URL, safer)

# GET

- Sends form data as part of the **URL query string**

Example:

`http://localhost/cgi-bin/script.py?name=John`

- Good for short, non-sensitive data.
- Data visible in browser history and server logs.
- **Size limit:** depends on browser/server (typically 2KB–8KB).
- Easy to test manually (just type in browser URL).



# POST

- Sends form data in the **HTTP request body**.
- Better for sensitive or large data (passwords, text areas).
- Data **not visible** in URL.
- Slightly more secure.
- Requires reading from `sys.stdin` (or handled automatically by `cgi.FieldStorage()`).

# What is GET Method?

- The **GET method** is one of the two main ways (GET & POST) a web browser can send data to a server.
- It **attaches data to the URL as query parameters**.
- The server (or CGI script) reads this data and sends back a response.

## Example

Suppose you have this HTML form:

```
<form action="/cgi-bin/hello.py" method="get"> Name: <input type="text"
name="name">
```

```
<input type="submit" value="Submit">
```

```
</form>
```

**Localhost:8080/form1.html**

If you type John in the box and click submit, the browser sends a request like this:

/cgi-bin/hello.py?name=John

Notice:

? → starts the query string

name=John → this is the data you submitted

**Localhost:8080/form1.html**

```
#!C:/Users/Acer/AppData/Local/Microsoft/WindowsApps/python.exe
#!/usr/bin/env python3
import cgi

print("Content-type: text/html\n")

form = cgi.FieldStorage()
name = form.getvalue("name")

print("<html><body>")
print(f"<h1>Hello, {name}!</h1>")
print("</body></html>")
```

## Key Points About GET

- **Data is visible in the URL** → good for search pages, but not for passwords.  
**Can be bookmarked** → the URL stores the data, so you can share the link.  
**Limited data size** → URL can't be too long (depends on browser/server).  
**Less secure** → since data is in the URL, don't use for sensitive info.

# What is POST Method?

- The **POST method** is another way a web browser sends data to a server.  
But unlike GET, it **sends data in the request body (hidden) — not in the URL.**

## Example

HTML form using POST:

```
<form action="/cgi-bin/hello.py" method="post">
```

```
 Name: <input type="text" name="name">
```

```
 <input type="submit" value="Submit">
```

```
</form>
```

**Localhost:8080/form2.html**



When you type John and click submit, the browser sends a request like this:

- POST /cgi-bin/Hello\_2.py Content-Type: application/x-www-form-urlencoded name=John

Notice:

- No data in the URL — it is sent **behind the scenes** in the request body.

**Localhost:8080/form2.html**

# Python CGI Script to Handle POST

```
1 #!C:/Users/Acer/AppData/Local/Microsoft/WindowsApps/python.exe
2 #!/usr/bin/env python3
3 import cgi
4
5 print("Content-type: text/html\n")
6
7 form = cgi.FieldStorage()
8 name = form.getvalue("name")
9
10 print("<html><body>")
11 print(f"<h1>Hello, {name}!</h1>")
12 print("</body></html>")
```

handles POST too.

## Key Points About POST

**Data is NOT visible in the URL** → safer for sensitive information (passwords, login data).

**No size limit** (practically much larger than GET) → good for file uploads or large forms.

**Cannot be bookmarked** because data is not in URL.

**Slightly slower** than GET due to extra data handling.

# **GET vs POST in action** with code:

Form\_3

Hello\_3

# Validating Form Inputs

When we say "**validate inputs**", we mean:

- **Check what the user typed before using it.**  
So we don't get wrong, empty, or dangerous data in our program.

# Why Validation is Important

Imagine a teacher collecting exam papers:

- Some students leave answers blank.
- Some write random things.
- Some write too long answers where only a number was expected.

The teacher **checks** them before marking.

This is exactly what our script should do with **form inputs**.

# Types of Validation

## 1. Check if the field is not empty

We don't want blank data.

if not text.strip():

```
 print("<h3 style='color:red'>Please enter some text!</h3>")
```

### Step 1: text.strip()

- strip() is a **string method** in Python.
- It removes **leading and trailing spaces** (and newline characters) from a string.

Example

```
" hello ".strip() # Output: "hello"
```

```
" ".strip() # Output: "" (empty string)
```

## Step 2: not text.strip()

- After stripping, if the result is an empty string "", it is considered **False** in Python.
- not turns it into **True**.
- This means the if condition becomes True **only when the user entered nothing or just spaces**.



## Step 3: Printing the Message

- `print("<h3 style='color:red'>Please enter some text!</h3>")`
- This sends an **HTML <h3> tag** back to the browser.
- `style='color:red'` makes the text red.
- The message is shown to the user as a **warning/error message**.

```
if not text.strip(): print("
```

```
Please enter some text!
```

```
")
```

# Create an HTML Form (empty\_form.html)

```
<!DOCTYPE html>
<html>
<head>
 <title>Comment Form</title>
</head>
<body>
 <h2>Enter Your Comment</h2>
 <form action="/cgi-bin/comment.py" method="post">
 <textarea name="comment" rows="4" cols="40"></textarea>

 <input type="submit" value="Submit">
 </form>
</body>
</html>
```

Save  
for

ot

# Create the CGI Script (empty\_form\_error.py)

```
1 #!C:/Users/Acer/AppData/Local/Microsoft/WindowsApps/python.exe
2 import cgi
3 # CGI header for browser
4 print("Content-type: text/html\n")
5 # Get data from form
6
7 form = cgi.FieldStorage()
8 text = form.getvalue("comment", "") # default = empty string if nothing entered
9
10 print("<html><body>")
11
12 # Check if text is empty (or just spaces)
13 if not text.strip():
14 print("<h3 style='color:red'>Please enter some text!</h3>")
15 else:
16 print(f"<h3 style='color:green'>You entered: {text}</h3>")
17
18 print("</body></html>")
```

unccctory.

## Run and Test

- Open a browser and go to:  
`http://localhost:8080/empty_form.html`
- Type something → click submit → you'll see your input displayed in green.
- Submit empty or just spaces → you'll see **red error message**.

## What's Happening

The HTML form **sends data** to comment.py using **POST method**.

The Python script reads the data using `cgi.FieldStorage()`.

`if not text.strip():` checks if input is empty → shows red message.

Otherwise, it displays the user's comment in green.

## 2. Check if the data type is correct

If we expect a number, make sure user typed a number.

When we take input from an HTML form in Python (using `cgi.FieldStorage()`), **everything comes as a string** — even numbers.

Example:

```
form.getvalue("age") # If user typed 25, this returns "25" (string)
```

If you want to use it in a calculation (like `age + 5`), you must convert it to an integer:

```
num = int("25") # Works fine, num = 25 (integer)
```

But if the user typed something wrong:

```
num = int("abc") # This will cause an error (ValueError)
```

So we must **check before using it** — this is where validation comes in.  
Imagine we have a form where the user must enter their age. (age.html)

```
<form action="/cgi-bin/validate_age.py" method="post">
 Enter your age: <input type="text" name="age">
 <input type="submit" value="Submit">
</form>
```

Enter your age:

# Python CGI Script (validate\_age)

```
1 #!C:/Users/Acer/AppData/Local/Microsoft/WindowsApps/python.exe
2 import cgi
3 print("Content-type: text/html\n")
4 form = cgi.FieldStorage()
5 print("<html><body>")
6 try:
7 # Try to convert the input into an integer
8 age = int(form.getvalue("age"))
9
10 # Extra check: make sure age is a sensible number
11 if age <= 0:
12 print("<h3 style='color:red'>Error: Age must be greater than 0!</h3>")
13 elif age > 120:
14 print("<h3 style='color:red'>Error: Age must be less than 120!</h3>")
15 else:
16 print(f"<h3 style='color:green'>Thank you! Your age is {age}.</h3>")
17 except:
18 # This block runs if user typed letters or left field empty
19 print("<h3 style='color:red'>Error: Please enter a valid number for age.</h3>")
20 print("</body></html>")
```



## What This Script Does

- Reads the value from the form.
- **Tries to convert** it into an integer (int()).
- If it fails → shows **red error message**.
- If conversion works → checks if age is within a valid range (1–120).
- Shows result in green if everything is fine.

# Sample Output (localhost:8080/age.html)

User Input	Result
25	"Thank you! Your age is 25." (green)
-5	"Error: Age must be greater than 0!"
200	"Error: Age must be less than 120!"
abc	"Error: Please enter a valid number for age."
(empty)	"Error: Please enter a valid number for age."

# What is “Length Validation”?

- When users type something in a form (like name, password, message), you often want to **control how short or long it can be**.
- If it's **too short** → it might not be a valid answer (e.g., name = “A”).
- If it's **too long** → it might be an error, spam, or cause issues in your database.

## Example: Checking Name Length

- Let's say we are asking a user for their name in a CGI form.

### HTML Form (length.html):

```
<form action="/cgi-bin/check_length.py" method="post">
 Enter your name: <input type="text" name="username">
 <input type="submit" value="Submit">
</form>
```

Enter your name:

# Python CGI Script (check\_length.py):

```
1 #!C:/Users/Acer/AppData/Local/Microsoft/WindowsApps/python.exe
2 import cgi
3
4 print("Content-Type: text/html\n")
5
6 form = cgi.FieldStorage()
7 name = form.getvalue("username", "")
8
9 # Length validation
10 if len(name) < 3:
11 print("<h3 style='color:red'>Error: Name is too short. Minimum 3 characters required.</h3>")
12 elif len(name) > 20:
13 print("<h3 style='color:red'>Error: Name is too long. Maximum 20 characters allowed.</h3>")
14 else:
15 print(f"<h3 style='color:green'>Welcome, {name}!</h3>")
```

- **len(name)** → counts how many characters are in the string.
- **if len(name) < 3:**

Means name is too short (less than 3 characters).

- **elif len(name) > 20:**

Means name is too long (more than 20 characters).

If both checks are fine, we accept the name.

User Input	Length	Result
A	1	<input type="checkbox"/> Too short → Error
Ram	3	<input type="checkbox"/> Accepted
JonathanSmithWithAVeryLongName	28	<input type="checkbox"/> Too long → Error

# What Does “Check Allowed Values” Mean?

Sometimes, you want users to choose **only certain values**.  
If they type anything outside the allowed list → show an error.

This is very useful when:

- You ask for **gender** (only Male, Female, Other allowed).
- You ask for **country code** (only IN, US, UK allowed).
- You ask for **rating** (only 1 to 5 allowed).



## Example: Gender Selection

- HTML Form (gender.html):

```
<form action="/cgi-bin/check_allowed.py" method="post">
 Select Gender:
 <select name="gender">
 <option value="">--Choose--</option>
 <option value="Male">Male</option>
 <option value="Female">Female</option>
 <option value="Other">Other</option>
 </select>
 <input type="submit" value="Submit">
</form>
```

## Example: Gender Selection

- HTML Form (gender.html):

Select Gender: --Choose-- ▾ Submit

--Choose--

Male

Female

Other

# Python CGI Script (check\_allowed.py):

```
1 #!C:/Users/Acer/AppData/Local/Microsoft/WindowsApps/python.exe
2 import cgi
3
4 print("Content-Type: text/html\n")
5
6 form = cgi.FieldStorage()
7 gender = form.getvalue("gender", "")
8
9 # Allowed values list
10 allowed_genders = ["Male", "Female", "Other"]
11
12 # Validation
13 if gender not in allowed_genders:
14 print("<h3 style='color:red'>Error: Invalid selection! Please choose a valid gender.</h3>")
15 else:
16 print(f"<h3 style='color:green'>You selected: {gender}</h3>")
```

## Explanation

- allowed\_genders is a **list of correct answers**.
- if gender not in allowed\_genders:
  - Checks if user's input is **NOT** in the list.
  - If not → show error.
- Else → accept and show success message.

# Client-Side vs Server-Side Validation

- **Client-Side** (done in browser)
  - Example: `<input type="number" required>`
  - Easy and fast — prevents submitting wrong data early.
  - But can be bypassed (hackers can disable it).
- **Server-Side** (done in Python script)
  - Example: if not text: ...
  - Must always do this — because it is the final check before using data.

## Key Points to Remember (Beginner Checklist)

- Always check if the input exists
- Convert to the right type (int, float)
- Handle empty strings safely
- Show **clear error messages** so users know what to fix

## Why This Is So Important

- Prevents **program crashes** (no "Internal Server Error").
- Prevents **wrong calculations** (e.g., adding words instead of numbers).
- Makes your site **user-friendly** (gives feedback).
- Adds **security** (prevents hackers from injecting bad data).

# **Web (HTTP) Servers with Python**

**Explain what an HTTP server does**

**Run a Python HTTP server**

**Serve static files (HTML, CSS, images)**

**Understand the basic flow of HTTP communication**

**Customize server responses using Python**

**Experiment with local development**

**Gain confidence to move toward dynamic web apps**



# What is an HTTP Server?

## Definition

- An **HTTP server** is a software (or sometimes hardware + software) that:
- **Listens** for requests from a web browser or client (on port 80 for HTTP, 443 for HTTPS).
- **Processes** the request.
- **Sends back a response** — usually an HTML page, image, JSON data, etc.
- It uses the **HTTP protocol** (Hypertext Transfer Protocol), which is the standard language of communication on the web.

## How it Works (Simple Steps)

- **Client (browser)** sends a request (like typing `https://example.com` in your browser).
- **DNS** finds the IP address of the server.
- **HTTP server** receives the request (e.g. "GET /index.html").
- The server finds the requested file (or runs a script if dynamic content is needed).
- **HTTP server sends back response** (HTML page + headers).
- The browser **renders the page** for you.

## Examples of HTTP Servers

- Apache HTTP Server (very popular)
- Nginx
- Microsoft IIS
- Lighttpd
- Built-in HTTP server in Python (for testing)

# Using Python's Built-in HTTP Server

## Step 1: Create a Folder

Make a folder anywhere on your computer, e.g.:

C:\Users\my\_website

Put some files inside it like:

- index.html (your web page)
- style.css
- image.png

```
C:\Users\Acer>cd my_website

C:\Users\Acer\my_website>python -m http.server 8000
Serving HTTP on :: port 8000 (http://[::]:8000/) ...
::1 - - [07/Sep/2025 16:50:42] "GET / HTTP/1.1" 200 -
```

## Step 2: Start the Server

`python -m http.server 8000`

- 8000 is the **port number** (you can choose another like 8080).
- Python will now serve your files.

```
C:\Users\Acer>cd my_website
```

```
C:\Users\Acer\my_website>python -m http.server 8000
```

```
Serving HTTP on :: port 8000 (http://[::]:8000/) ...
```

```
:::1 - - [07/Sep/2025 16:50:42] "GET / HTTP/1.1" 200 -
```

## Step 3: Open in Browser

- Go to:
- `http://localhost:8000`
- You will see a **directory listing** of your files.  
If there's an `index.html`, it will open automatically.

## Mini Project: Serve Custom HTML (No Directory Listing)

- Instead of showing all files in the folder, we can **override the default behavior** and always return a custom page.

- Python's built-in `http.server.SimpleHTTPRequestHandler` will **show a directory listing** when you visit a folder that does **not** contain an `index.html`.

To stop that and always return your own custom page instead, you can:

- **Easy method:** Put an `index.html` in the folder (then there's no listing).
- **Programmatic method (recommended for learning):** subclass `SimpleHTTPRequestHandler` and **override** the method that creates the directory listing so it returns your custom HTML.



## Step 1: The Normal Way (Default Behavior)

- When you run this command inside a folder:
- `python -m http.server 8000`
- Python starts a **small web server** on your computer.  
When you open `http://localhost:8000` in your browser, Python will show:
- **index.html** if it exists.
- Otherwise, a **list of all files in the folder** (directory listing).

Example of what you might see without `index.html`:

- Directory listing for /
- `index.html`
- `style.css`
- `image.png`

## Step 2: Why We Override

- Sometimes we **don't want people to see a list of files**. Instead, we want to **always show a custom page** like:

"Welcome to my website!"

- even if `index.html` doesn't exist.
- To do this, we **override** the part of Python's server that shows the file list.  
That part is called `list_directory`.

## Step 3: A Small Working Example

- Here's a **very simple version** you can copy-paste and run — much easier to understand:

### 1. Create a folder named `my_website`

- Inside it, create one file:

`custom_index.html` (already saved in `my_website`)

```
<!DOCTYPE html>
<html>
<head>
 <title>My Custom Page</title>
</head>
<body style="text-align:center; font-family:Arial">
 <h1>Welcome to My Website</h1>
 <p>This page is shown instead of directory listing.</p>
</body>
</html>
```

## 2. Create a Python file outside the folder

### serve\_custom.py

```
1 from http.server import HTTPServer, SimpleHTTPRequestHandler
2 import os
3
4 # Change working directory to the folder we want to serve
5 os.chdir("my_website")
6
7 class NoListingHandler(SimpleHTTPRequestHandler):
8 def list_directory(self, path):
9 # Open our custom page instead of showing file list
10 with open("custom_index.html", "rb") as f:
11 content = f.read()
12
13 # Send the page to the browser
14 self.send_response(200)
15 self.send_header("Content-Type", "text/html")
16 self.send_header("Content-Length", str(len(content)))
17 self.end_headers()
18 self.wfile.write(content)
19 return None
20
21 # Start the server
22 server = HTTPServer(("localhost", 8000), NoListingHandler)
23 print("Serving on http://localhost:8000")
24 server.serve_forever()
```

### 3. Run and Test

- Open terminal in the folder where `serve_custom.py` is saved:
- `python serve_custom.py` (command prompt)
- Go to <http://localhost:8000> →

You will **always see your custom page**, never a file list.

- This is the **entire idea**:
- Normally, Python shows file listing.
- We tell Python: "Don't list files. Just send my custom page."

# Custom Web Servers

- How to use BaseHTTPRequestHandler
- How to inspect self.path to serve different content
- How to send custom responses (HTML, text, data)
- How to create your own mini web framework

## What You Learned

- **http.server module** → A built-in Python module that allows you to quickly create a simple web server.
- **Serving static files** → You can use it to serve HTML, CSS, images, or any files stored in a folder.



```
1 # Run this in the folder where your HTML files are
2 import http.server
3 import socketserver
4
5 PORT = 8000
6 handler = http.server.SimpleHTTPRequestHandler
7
8 with socketserver.TCPServer(("", PORT), handler) as httpd
9 print(f"Server running at http://localhost:{PORT}")
10 httpd.serve_forever()
```

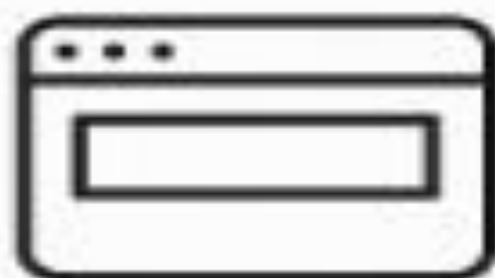
---

**Key Code Example (server.py already stored in user→Acer)**  
**command prompt→python server.py**

**Starts the local server**

**Localhost:8000 in Browser**

- When you run this code, Python starts a small web server.
- You can open <http://localhost:8000> in your browser and see your files.
- By default, it shows a **directory listing** (list of files) unless you have an **index.html** file (then it shows that page).
- Instead of just listing files, you can make the server send a **custom HTML page**



**Prebuilt server**



Yesterday = you used a **ready-made waiter** that always gives a menu.  
Today = you become the **chef**, deciding exactly what to serve when someone asks.

Now we go one step deeper — instead of using SimpleHTTPRequestHandler, we will **write our own request handler** using BaseHTTPRequestHandler.  
This gives you full control over what your server does when a browser requests a page.

# 1. What is BaseHTTPRequestHandler?

- It is a **Python class** that handles web requests.
- When someone opens your server in a browser, a **request** is sent.
- You can decide:
  - What message to send back.
  - Whether to show text, HTML, or even time/date.

# Step 1: Create custom\_server.py

```
1 from http.server import HTTPServer, BaseHTTPRequestHandler
2 class MyHandler(BaseHTTPRequestHandler): # Our custom request handler
3 def do_GET(self): # This function runs when the browser sends a GET request
4 self.send_response(200) # 200 = OK (successful)
5 self.send_header("Content-type", "text/html") # Tell browser we are sending HTML
6 self.end_headers()
7 # Send a message back to browser
8 self.wfile.write(b"<h1>Welcome to My Server</h1>") # b = convert to bytes
9 # Create server (host, port)
10 server_address = ("", 8000)
11 httpd = HTTPServer(server_address, MyHandler)
12 print("Server is running at http://localhost:8000")
13 httpd.serve_forever()
```

## **Step 2: Run the Server**

- Open Command Prompt where the file is saved and run:
- `python custom_server.py`

## Step 3: Open in Browser

Go to:

- `http://localhost:8000`

You should see:

- **Welcome to My Server**



## **Step 4: Add a Special Page /time**

Let's make it smarter — if someone types /time, we show the current time.

```
1 from http.server import HTTPServer, BaseHTTPRequestHandler
2 import datetime # To get current time
3 class MyHandler(BaseHTTPRequestHandler):
4 def do_GET(self):
5 if self.path == "/time": # If user typed /time in browser
6 now = datetime.datetime.now()
7 message = f"<h1>Current Time: {now.strftime('%Y-%m-%d %H:%M:%S')}</h1>"
8 else:
9 message = "<h1>Welcome to My Server</h1>"
10
11 self.send_response(200)
12 self.send_header("Content-type", "text/html")
13 self.end_headers()
14 self.wfile.write(message.encode("utf-8")) # Convert to bytes
15
16 server_address = ("", 8000)
17 httpd = HTTPServer(server_address, MyHandler)
18 print("Server is running at http://localhost:8000")
19 httpd.serve_forever()
```

## Now Test Again:

- Open `http://localhost:8000` → shows "Welcome to My Server"
- Open `http://localhost:8000/time` → shows **current system time**

## Key Idea for Beginners

- `do_GET()` = **What to do when someone visits your server.**
- `self.path` = **What part of the website they are asking for (/ or /time).**
- You can send **different HTML** depending on the path.

This is how websites have **different pages** like /about, /contact, /products.

# Flow

- You open your browser and type `http://localhost:8000/time`
- Browser sends request → Python server receives it
- Python checks:
  - Did user ask for `/time`? → Yes → Send back current time
  - Otherwise → Send "Welcome to My Server"
- Browser displays what Python sent.

# What's the Difference?

Feature	http.server (Simple Server)	BaseHTTPRequestHandler (Custom Server)
What we used	http.server module + SimpleHTTPRequestHandler	http.server module + BaseHTTPRequestHandler
Purpose	Serve files quickly (HTML, CSS, images)	Write your own logic for what to send back
Default Behavior	Shows a directory listing OR index.html automatically	Does nothing until YOU tell it what to send
Control	Limited – just serves files from folder	Full control – you decide what happens for /, /time, etc.
Example Output	Shows contents of folder automatically	Can show any message, like "Welcome to My Server"
Use Case	Quick local testing of a static website	Building a custom web application or API

# **Integrating CGI with Web Server**

Right now, you are learning **how to make your Python server run small programs** (called **CGI scripts**) and show their output on a web page.

This means:

- You make a **form** in HTML (where user types something).
- When user clicks **submit**, the server runs a **Python program** to process that input.
- The output (result) is sent back and displayed in browser.
- This is the first step toward making **dynamic websites** (not just static pages).

## Example:

- We want to build a **Word Counter**:
- User opens a page that shows a text box.
- User types some text (for example: *Hello how are you*).
- User clicks **Count Words**.
- Our server runs a Python script that counts words.
- The result is shown (like *You typed 4 words*).



# Step by Step

## Step 1: Start a CGI-enabled Server (server\_1)

- Here is a **tiny server** that can run CGI scripts:

```
1 # server_1.py
2 from http.server import HTTPServer, CGIHTTPRequestHandler
3
4 # Create a server that runs on port 8000
5 server_address = ("", 8000)
6 httpd = HTTPServer(server_address, CGIHTTPRequestHandler)
7
8 print("CGI Server running at http://localhost:8000")
9 httpd.serve_forever()
```

## Step 2: Make Sure You Have a cgi-bin Folder

- Inside your project folder, create this structure:
- my\_project/
  - server\_1.py (your server code)
  - index\_1.html (form)
  - cgi-bin/ word-count.py (script to run when form is submitted)
- **Rule:**  
CGI scripts must go inside cgi-bin folder, otherwise the server won't run them.

## Step 3: Make the HTML Form (index\_1.html)

- This is the page that user will open:

```
<!DOCTYPE html>
<html>
<head>
 <title>Word Counter</title>
</head>
<body>
 <h1>Enter Your Text</h1>
 <form action="/cgi-bin/wordcount.py" method="post">
 <textarea name="usertext" rows="5" cols="40"></textarea>

 <input type="submit" value="Count Words">
 </form>
</body>
</html>
```

## Step 3: Make the HTML Form (index\_1.html)

### What happens here?

- `<form action="/cgi-bin/word-count.py" method="post">`  
means:  
“When user clicks submit, run wordcount.py file from cgi-bin folder.”

## Step 4: Write the CGI Script (cgi-bin/word-count.py)

```
1 import cgi
2 # 1. Required: Tell browser we are sending HTML back
3 print("Content-Type: text/html")
4 print() # blank line is required
5 # 2. Read input data from form
6 form = cgi.FieldStorage()
7 text = form.getvalue("usertext", "")
8 # 3. Process input
9 words = text.split()
10 count = len(words)
11 # 4. Show result
12 print("<html><body>")
13 print("<h1>Word Count Result</h1>")
14 print(f"<p>You typed: {count} words</p>")
15 print("</body></html>")
```

## **This script:**

- Reads the text user entered
- Splits it into words
- Counts them
- Sends back a small HTML page with the result

## Step 5: Run and Test

- Open terminal, run:  
`python server_1.py`
- Open browser → go to:  
`http://localhost:8000/index_1.html`
- Type some text → click **Count Words**
- You should see result page with number of words.

# Beginner-Friendly Flow Diagram

Imagine it like a restaurant

- **Browser** = Customer
- **Form** = Menu Card (user chooses something)
- **Server** = Waiter
- **CGI Script** = Chef (cooks the result)
- **Result Page** = The meal served back to customer

So:

- Browser sends input → Server → Chef (CGI script)
- Chef prepares result → Server sends back → Browser shows it